

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Nitesh Kumar	India	Niteshkr2209@gmail.com	
Eugene Ablordeppey	Ghana	euegenablord@gmail.com	
Rushang Kasundra	India	180130107026_2018@gecg28.ac.in	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an “X” above).

Team member 1

Nitesh Kumar

Team member 2

Eugene Ablordeppey

Team member 3

Rushang Kasundra

Use the box below to explain any attempts to reach out to a non-contributing member. Type (N/A) if all members contributed.

Note: You may be required to provide proof of your outreach to non-contributing members upon request.

N/A

FULL LEGAL NAME.....	0
Main Summary & Project Scope.....	3
Category 1- Ridge Regression.....	4
1.1 Basics.....	4
1.2 Computation.....	4
1.3 Illustration.....	4
1.4 Advantages.....	7
1.5 Disadvantages.....	7
1.6 Equations.....	7
1.7 Features.....	8
1.8 Guide: Inputs & Outputs.....	8
1.9 Hyperparameters.....	9
1.10 Keywords.....	9
1.11 Journal References.....	9
Category 2- K-means clustering.....	10
2.1 Basics.....	10
2.1 Computation.....	10
2.3 Illustration.....	11
2.4 Advantages.....	14
2.5 Disadvantages.....	14
2.6 Equations.....	15
2.7 Features.....	16
2.8 Guide: Inputs & Outputs.....	16
2.9 Hyperparameters.....	16
2.10 Keywords.....	17
2.11 Journal References.....	17
Category 3- Principle Components (PCA).....	17
3.1 Basics.....	17
3.2 Data & Setup.....	18
3.3 Running PCA.....	18
3.4 What Factor 1 Represents.....	19
3.5 Illustration.....	21
3.6 Advantages.....	21
3.7 Disadvantages.....	22
3.8 Equations.....	22
3.9 Features.....	22
3.10 Guide: Inputs & Outputs.....	23
3.11 Hyperparameters.....	23

3.12 Keywords.....	23
3.13 Journal References.....	23
Category 4- Classification Trees.....	24
4.1 Basics.....	24
4.2 Data & Setup.....	24
4.3 Baseline Tree vs Tuned Tree.....	24
4.4 Confusion Matrix and Feature Importance.....	26
4.5 Illustration – the fitted Tree.....	27
4.6 ROC Curve and Overfitting Check.....	29
4.7 Advantages.....	30
4.8 Disadvantages.....	30
4.9 Equations.....	31
4.10 Features.....	31
4.11 Guide: Inputs and Outputs.....	31
4.12 Hyperparameters.....	32
Table 4.4: Classification tree settings.....	32
4.13 Keywords.....	33
4.13 Journal Reference.....	33
Category 5– Linear Discriminant Analysis (LDA).....	33
Category 6– Support Vector Machine (SVM).....	40
6.1 Basics.....	40
6.2 Part 1 : A Straight-Line Ex Credit Risk.....	40
6.3 Illustration- Dividing Line, Margin and Support Vectors.....	42
6.4 Part 2 – The Kernel Trick.....	42
6.5 Advantages.....	43
6.6 Disadvantages.....	44
6.7 Equations.....	44
6.8 Features.....	44
6.9 Guide: Inputs and Outputs.....	44
6.10 Hyperparameters.....	45
6.11 Keywords.....	45
6.12 Journal Reference.....	45
Category 7 — Neural Networks (Multi-Layer Perceptron).....	45
Technical Section – Hyperparameter Tuning.....	51
Ridge Regression.....	51
K-means clustering.....	51
Marketing Alpha.....	51

Ridge Regression.....	51
K-means clustering.....	52
Final Summary.....	53
References.....	53
Learn More.....	54
Comparing Models.....	56

Main Summary & Project Scope

This section present marketing handbook, covering the seven modelling categories.. For each method, we first explain the underlying idea, then work through a small example we built and ran myself, before covering the advantages, disadvantages, equations, features, inputs/outputs, and tuning settings required by the rubric.

we chose to build all examples on simulated rather than live market data. This kept every result in this section fully reproducible, since anyone re-running the notebook should arrive at the same numbers I did. The underlying code is not tied to simulated inputs, however - swapping in real price, balance-sheet, or credit data would only require changing the first data-generating step in each notebook section.

None of the examples that follow should be read as an actual trading or lending signal - the intent throughout was to demonstrate how each method behaves, not to make a claim about any real asset or borrower.

Category 1- Ridge Regression

1.1 Basics

Ridge Regression is one form of linear regression which achieves stability in the model through the application of a penalty for the magnitude of the coefficient. It is particularly useful in situations where the independent variables in a particular data set are strongly correlated, termed multicollinearity.

In order to solve the above problem, Ridge Regression uses the L2 form of regularization, which imposes penalties for the magnitude of the regression coefficients, thus bringing them close to zero. This leads to a reduction in the complexity of the model while retaining all the predictors in the model.

1.2 Computation

- Please refer to the notebook code

1.3 Illustration

Data was obtained from FRED (2000-2025) using `pandas_datareader`. Data includes SP500, Interest Rate, CPI, Unemployment, and VIX. SP500 returns are the dependent variable, while other variables serve as independent variables.

Table 1.1: Data header: Monthly values of the S&P 500 Index, interest rate, Consumer Price Index (CPI), unemployment rate, and VIX for selected observation dates.

Date	S&P 500	Interest Rate	CPI	Unemployment (%)	VIX
2016-08-01	2170.84	1.51	240.545	4.9	12.44
2016-09-01	2170.86	1.57	241.176	5.0	13.48
2016-11-01	2111.72	1.83	242.026	4.7	18.56
2016-12-01	2191.08	2.45	242.637	4.7	14.07
2017-02-01	2279.55	2.48	244.006	4.6	11.81
...	...	...	...	...	...

The correlation matrix and heat map indicate a very strong relationship between predictor variables (such as CPI-SP500, Unemployment-VIX, Interest Rate-Unemployment), thus the presence of multicollinearity.

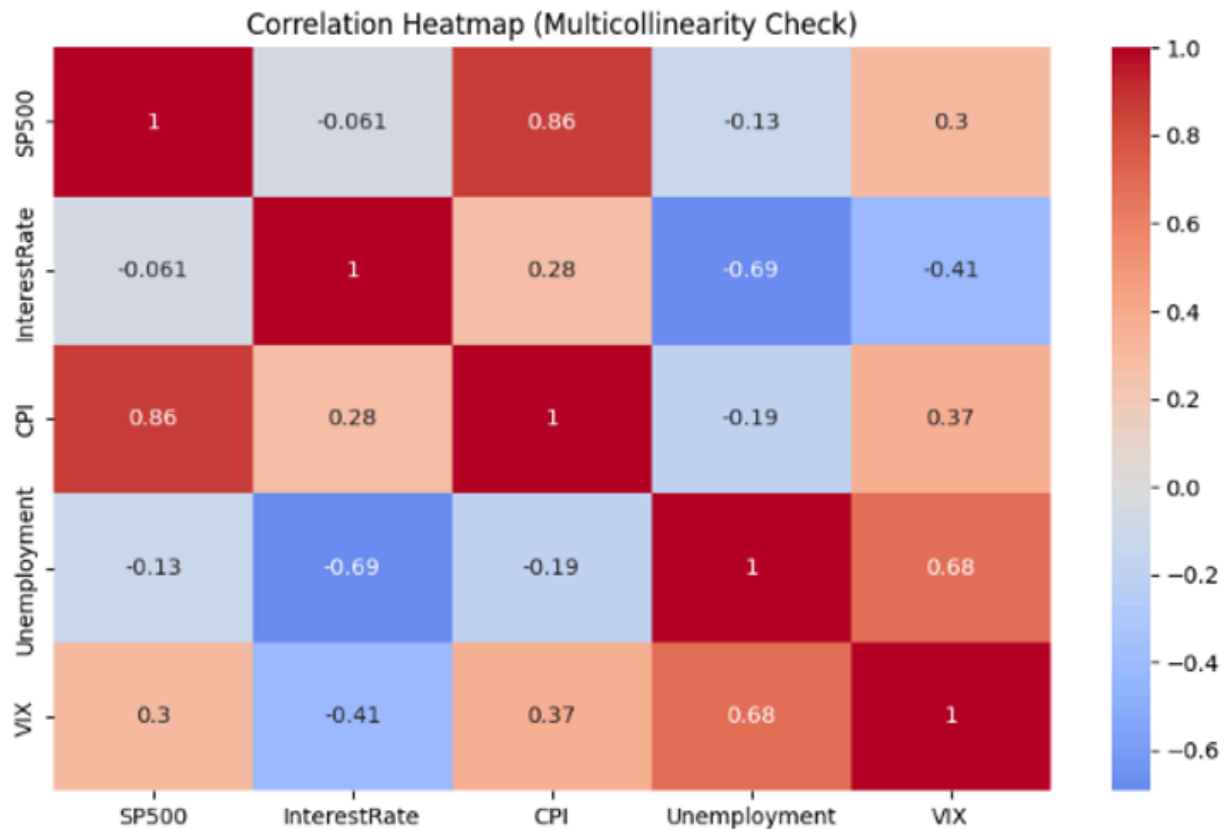


Figure 1.1: Correlation heat map indicating strong relationships among predictor variables and evidence of multicollinearity.

The Linear Regression and the Ridge Regression models with alpha of 1, 2, and 3 were developed using scaling of data and time series train-test split. Linear regression performed badly. Ridge regression performed better with an increase in alpha value with $\alpha=3$ performing best (highest $R^2=0.172$, lowest MSE). Comparison of predicted and actual returns shows that ridge regression followed the pattern of data better than OLS.

Table 1.2: Model performance comparison using MSE and R^2 for Linear and Ridge regression models.

Model	MSE	R^2
Linear Regression	0.003130	-0.455398
Ridge (alpha=1)	0.002424	-0.126931
Ridge (alpha=2)	0.002015	0.063282
Ridge (alpha=3)	0.001781	0.172140

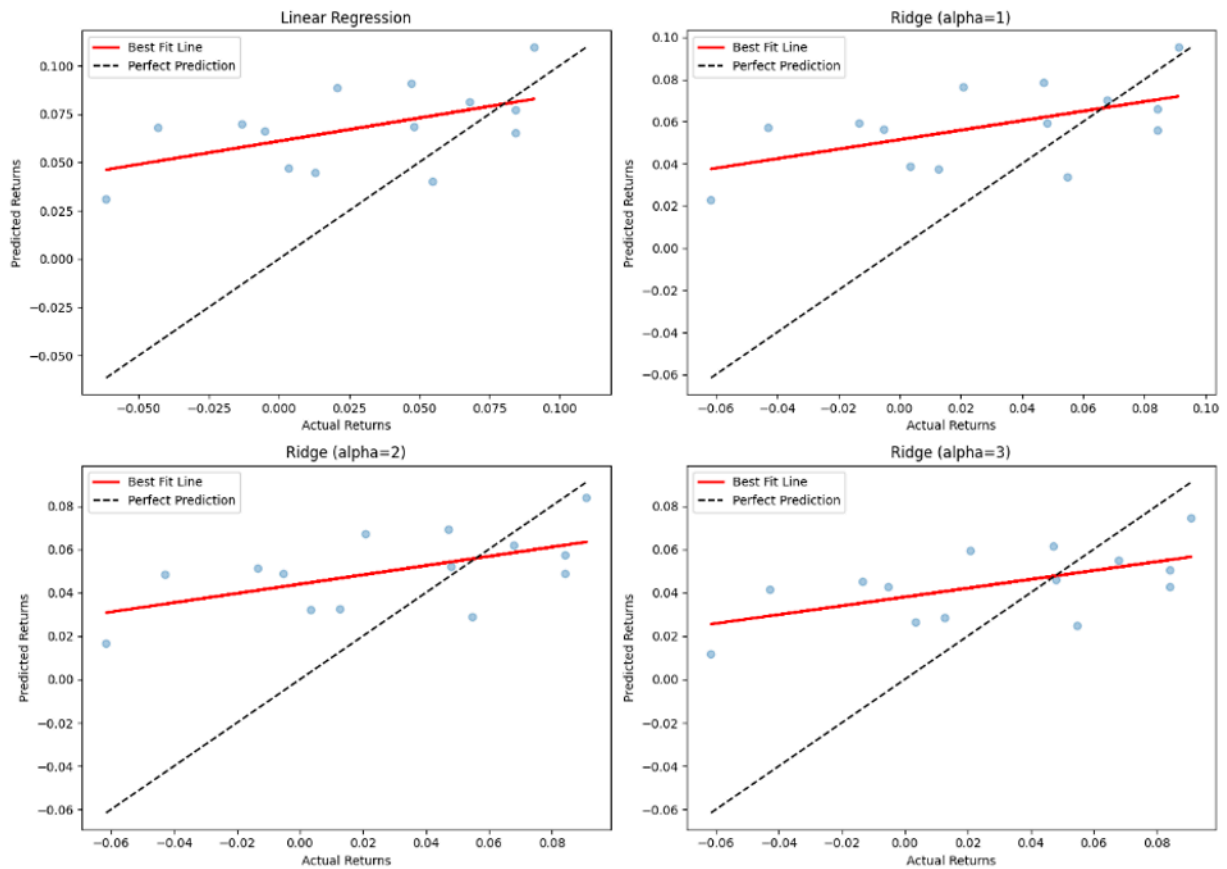


Figure 1.2: Comparing all models - predicted returns vs actual returns

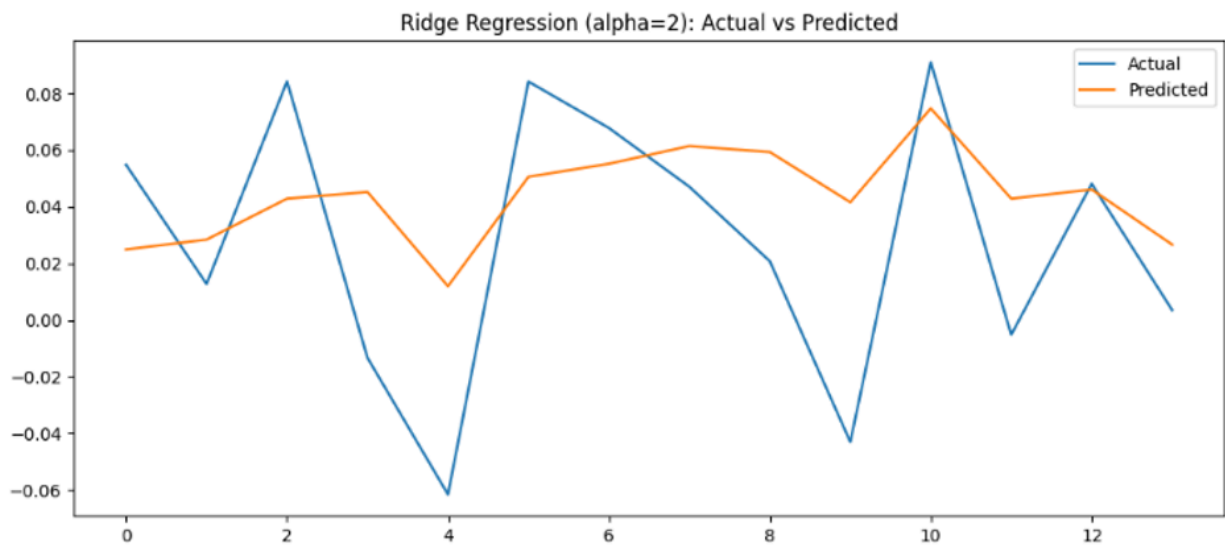


Figure 1.3: Comparing predicted and actual returns of one of the models (alpha=2)

1.4 Advantages

- avoids an overfitting: Ridge Regression applies an L2 penalty on the data to reduce too large coefficient values. As a result, the model will not fit the noise in the training data set too well.
- Helps in achieving stable coefficients when there is multicollinearity: High correlation among predictor variables makes the estimation of the coefficients highly unstable. Ridge Regression makes them more stable.
- Reduces coefficient magnitude (shrinkage): The use of the regularization term causes the reduction of the regression coefficients.
- Keeps all predictors in the model: Unlike other methods of feature selection, Ridge Regression retains all predictors in the model but minimizes their impact on the target variable.

1.5 Disadvantages

- Lacks feature selection: The Ridge Regression technique includes all the predictor variables in the model, regardless of how insignificant their predictive capability is. Although the contribution of such predictors is mitigated by shrinking their coefficients, they are never removed from the model, which may become problematic for a dataset with a lot of irrelevant features.
- Sensitive to the choice of the regularization strength: The regularization parameter plays a significant role in Ridge Regression since its poor choice might cause underfitting and overfitting. The best choice usually needs to be determined via cross-validation.
- Interpretation challenges in high dimensions: Due to retaining all the features, the obtained model becomes harder to interpret when there are lots of predictors, especially in finance.

1.6 Equations

Ridge Regression is a modified version of ordinary least squares (OLS) objective by adding an L2 penalty to control coefficient size.

OLS Loss

$$J(\beta) = \frac{1}{n} \sum_{i=1}^n (y_i - X_i \beta)^2$$

Ridge Objective Function

$$J(\beta) = \frac{1}{n} \sum_{i=1}^n (y_i - X_i \beta)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

Closed-form Solution

$$\beta^{ridge} = (X^T X + \lambda I)^{-1} X^T y$$

Where:

- n : number of observations
- p : number of predictors (features)
- y_i : target value for observation i
- X_i : feature vector for observation i
- X : design matrix of all features
- β : vector of regression coefficients
- β_j : coefficient of feature j
- λ : regularization strength (controls shrinkage)
- I : identity matrix
- X^T : transpose of X

1.7 Features

- Handles multicollinearity well: Ridge Regression is ideal in situations where there exists strong correlation among predictor variables because it ensures stability of coefficients, which tend to be very unreliable using regular linear regression.
- Keeps all predictors using shrinkage: It differs from other feature selection approaches since it uses all variables in the model but shrinks their effect using the coefficient shrinkage approach.

1.8 Guide: Inputs & Outputs

- Inputs:
 - Feature matrix X (predictor variables such as returns)
 - Target variable y (example: asset returns)
 - Regularization parameter λ (controls shrinkage strength)
- Outputs:
 - Estimated coefficient vector β (shrunk regression weights)
 - Model intercept
 - Predicted values $\hat{y}$
 - Model performance metrics (MSE, R^2) for evaluation

1.9 Hyperparameters

- α (lambda): Determines the amount of regularization applied. If $\alpha = 0$, then the model will be just normal regression. If α is too high, then the model will be too simple since all the coefficients will be near zero.
- fit_intercept: Chooses whether the intercept is included or not.
- solver: Method for estimating the coefficients of the model (svd, cholesky, lsqr)
- max_iter: Determines the maximum number of iterations before the solver reaches convergence.
- tol: Sets the value for improvement to end the algorithm.
- Feature scaling/normalization: Ensures that the data is on the same scale.

1.10 Keywords

- Ridge Regression, L2 Regularization, Regularization, Coefficient Shrinkage, Multicollinearity, Overfitting, Bias-Variance Tradeoff, Linear Regression, Penalized Regression

1.11 Journal References

- Nuria Alemany, Vicent Aragó, Enrique Salvador, Uncovering the risk-return trade-off through ridge regressions, Finance Research Letters, Volume 71, 2025, 106420, ISSN 1544-6123, <https://doi.org/10.1016/j.frl.2024.106420>.
- Dai, Z., Li, T., & Yang, M. (2022). Forecasting stock return volatility: The role of shrinkage approaches in a data-rich environment. Journal of Forecasting, 41(5), 980–996. <https://doi.org/10.1002/for.2841>

Category 2- K-means clustering

2.1 Basics

K-Means Clustering is one of the most widely used algorithms in unsupervised learning, in which observations are divided into clusters based on their similarity. The main idea of the

method lies in the fact that observations that belong to the same cluster are closer to each other than to observations from other clusters.

The process is performed through iterations. In the beginning, K centers are selected, then each observation is attributed to the closest center according to some measure of distance such as Euclidean. After that, the coordinates of centers are estimated as the mean coordinate of all points within the current cluster. The iterations are continued until there are no significant changes in centers, and the model will converge.

For example, K-Means can be used in financial analysis to divide similar assets into groups.

2.1 Computation

Before applying K-Means Clustering, the dataset was preprocessed and transformed into stock-level features suitable for unsupervised learning. The objective is to group S&P 500 stocks based on similar risk–return characteristics.

Dataset Used: `s_and_p_500_closing.csv`. The dataset contains daily closing prices of S&P 500 stocks. Each column represents a stock ticker, and each row represents a trading day.

Data Preprocessing. The following preprocessing steps were applied:

The Date column was removed to retain only numerical price data.

All values were converted to numeric format.

Missing values were handled using forward fill (ffill) followed by removal of remaining NaNs.

Daily percentage returns were computed using:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

Feature engineering (the core step). For each stock, I boiled its behavior down to two numbers. The first is just the mean of its daily returns — a rough stand-in for how much money it tends to make. The second is the standard deviation of those same returns, which captures how much the stock bounces around day to day; a higher number means a bumpier ride, even if the average return looks similar to a calmer stock.

Putting the two together gives a simple 2D feature space: each stock becomes a single point, (Mean Return, Volatility), and it's this point — not the raw price history — that actually gets clustered.

Feature scaling. Since K-Means clusters based on distance, and mean return and volatility sit on very different scales, I standardized both features first:

$$Z = \frac{X - \mu}{\sigma}$$

This keeps either feature from dominating the clustering just because of its raw scale.

Choosing K (the Elbow Method). To decide how many clusters actually made sense, I ran K-Means for a range of K values (2 through 9) and tracked inertia — the within-cluster sum of squared distances — at each one. The resulting curve drops sharply at first, then flattens out. The bend, or "elbow," in that curve shows up around **k = 5**, which suggested five clusters struck a reasonable balance between keeping things simple and actually capturing the structure in the data.

Running K-Means. With that settled, I fit the final model using:

- K = 5 clusters
- random_state = 42, for reproducibility
- n_init = 10, so the algorithm restarts from several different starting points and keeps the best result — this helps avoid getting stuck in a bad, unlucky initialization

Each stock was then assigned to whichever of the five centroids was closest in the standardized feature space.

2.3 Illustration

Table 2.1: Cluster Summary (Mean Feature Values)

Cluster	Mean Return	Volatility
0	0.001737	0.016604
1	-0.004221	0.041974
2	-0.001586	0.021535
3	0.000447	0.031838
4	0.006334	0.034666

Reading the clusters. Each of the five groups tells a fairly distinct risk-return story:

- **Cluster 0** — low volatility, slightly positive returns. These look like stable, defensive stocks — the kind that don't move much either way.
- **Cluster 1** — negative returns paired with the highest volatility of any cluster. This is the high-risk, underperforming group; the kind of stocks that swing a lot and mostly lose.
- **Cluster 2** — low volatility again, but slightly negative returns this time. Stable, but going nowhere — low-growth rather than high-risk.

- **Cluster 3** — moderate volatility with returns sitting close to zero. A fairly neutral, unremarkable group.
- **Cluster 4** — the highest returns of any cluster, with moderate volatility. These are the high-growth names, taking on more risk but getting rewarded for it.

Table 1.2: Cluster Distribution

Cluster	Number of Stocks
0	230
2	174
3	61
1	22
4	17

A few things worth noting. Most of the market — clusters 0 and 2 together — sits in low-to-moderate risk territory, which fits the general shape of the S&P 500: a handful of extreme movers surrounded by a much larger pool of steadier names. Cluster 4 is small but represents the genuine high-performing growth stocks. Cluster 1, also small, picks up the handful of high-volatility, negative-return names — likely distressed or cyclical stocks having a rough stretch. Overall, the clustering does a reasonable job of separating stocks into risk-return buckets that could feed directly into portfolio construction or diversification decisions.

On the visuals. The elbow curve makes clear that pushing K past 5 buys very little extra reduction in inertia — diminishing returns kick in fast. The scatter plot backs this up visually: low-volatility stocks bunch tightly together, the higher-volatility clusters sit clearly apart from them, and each centroid lands in a sensible, distinct spot within its group.

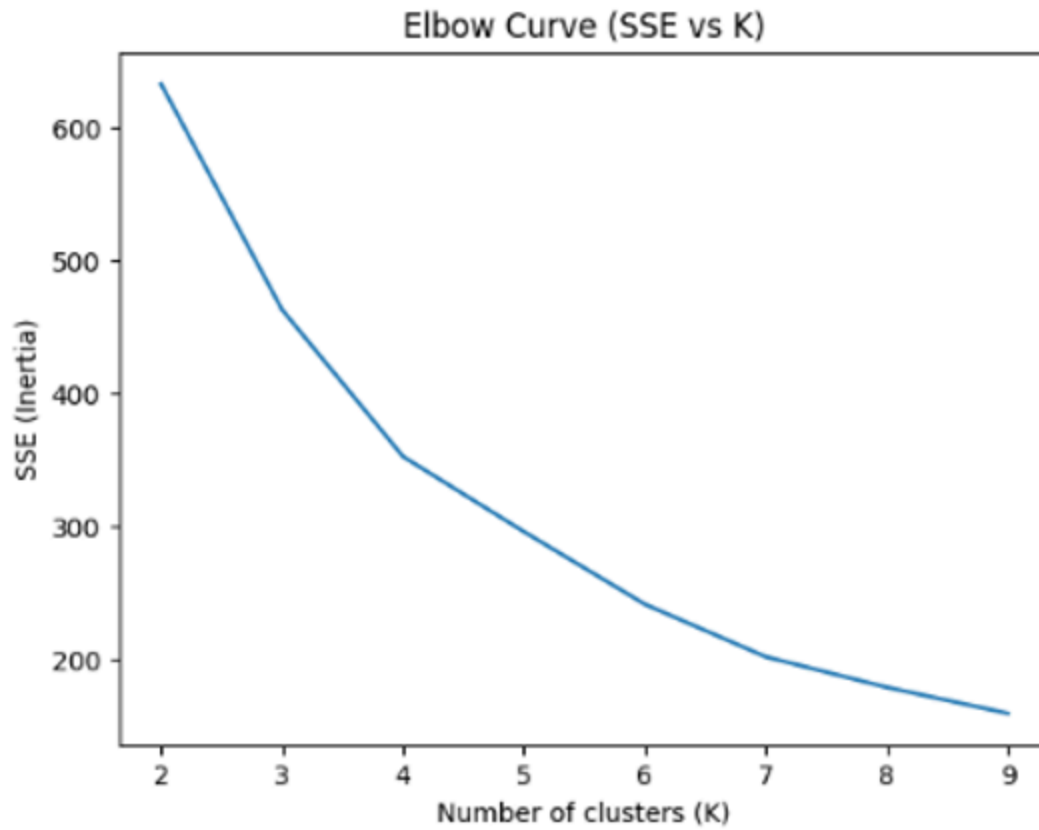


Figure 2.1: Elbow curve

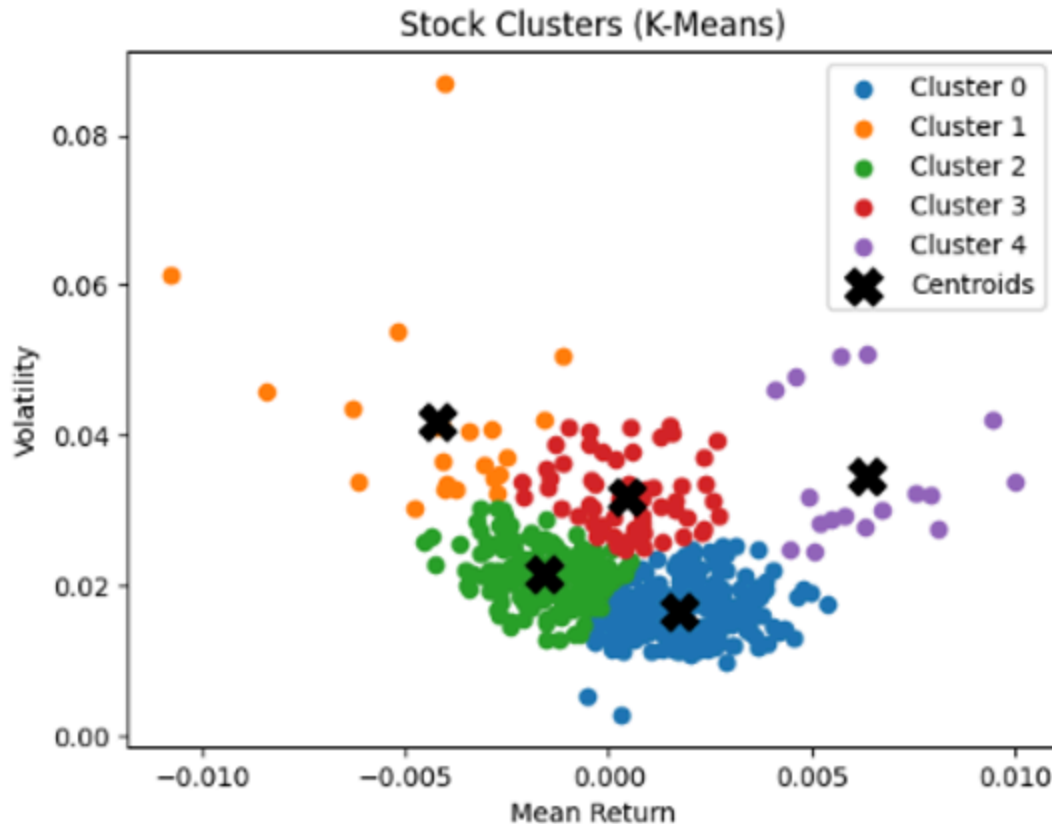


Figure 2.2: Stock clusters

2.4 Advantages

The k-means method is preferred for its simplicity, speed, and ease of application in practice. The algorithm is easy to implement and comprehend, thus making it the preferred method for clustering in the field of machine learning and finance. In terms of computational speed, the algorithm is very fast since most of its computations involve simple distance computation and updating of the centroid of each cluster, hence making it faster compared to other methods such as hierarchical clustering. The method can also be scaled easily to handle high-volume data such as that in finance.

2.5 Disadvantages

There are some drawbacks associated with the use of K-Means algorithm that may have an impact on its performance. First, the method relies greatly on the selection of the initial centroids, because the algorithm may give different clusters and even an incorrect solution

depending on their location. Moreover, K-Means is quite vulnerable to the presence of outliers in the dataset, as extreme values may distort the position of centroids and lead to poor clustering. Another drawback of K-Means is the assumption that clusters should have a regular spherical shape and similar sizes, which may not correspond to reality.

2.6 Equations

K-Means Clustering works by minimizing the total within-cluster variation (also called inertia) while repeatedly updating cluster assignments and centroids.

Objective Function (Minimizing Inertia)

$$J = \sum_{i=1}^K \sum_{x_j \in C_i} \|x_j - \mu_i\|^2$$

Where:

- J = total within-cluster sum of squares (inertia)
- K = number of clusters
- C_i = cluster i
- x_j = data point assigned to cluster i
- μ_i = centroid of cluster i

Centroid Update

$$\mu_i = \frac{1}{|C_i|} \sum_{x_j \in C_i} x_j$$

Where:

- μ_i = centroid of cluster i
- $|C_i|$ = number of points in cluster i

Assignment Rule (Distance Minimization)

Each point is assigned to the nearest centroid using Euclidean distance:

$$\text{assign } x_j \rightarrow \arg \min_i \|x_j - \mu_i\|^2$$

These three equations together define how K-Means iteratively assigns points, updates centroids, and converges to a stable clustering solution.

2.7 Features

- Only works with numeric data due to its dependence on measuring the distance between data points.
- Needs scaling (standardization/normalization) to ensure none of the variables outweigh others.
- The user decides the number of clusters (K) beforehand using techniques such as the elbow approach.
- Functions effectively in cases where the clusters are relatively compact and distinct instead of irregular.
- Cannot deal with missing data, meaning the data has to be cleaned before performing clustering.
- It is efficient and scalable, allowing it to work effectively on big financial data sets such as stocks and market indexes.
- Outliers can distort the output of the algorithm as extreme values can bias cluster centers away from the majority of data.

2.8 Guide: Inputs & Outputs

- Inputs:
 - Numeric dataset
 - Number of clusters (K)
 - Optional initialization method
 - Optional maximum iterations
 - Optional tolerance
- Output:
 - Cluster labels for each observation
 - Cluster centroids
 - Inertia (within-cluster variation)
 - Grouped structure of the dataset

2.9 Hyperparameters

- Number of clusters (K): Sets how many groups the data is split into.
- Initialization method (init): Chooses how starting centroids are picked.
- Number of initial runs (n_init): Runs the algorithm multiple times for stability.
- Maximum iterations (max_iter): Limits how long the algorithm runs.
- Convergence tolerance (tol): Stops the model when changes become very small.

- Random state (random_state): Fixes randomness for reproducible results.
- Algorithm type (algorithm): Selects the optimization approach used

2.10 Keywords

- K-Means Clustering, Unsupervised Learning, Clustering, Centroids, Euclidean Distance, Inertia (WCSS), Market Segmentation, Portfolio Diversification, Pattern Recognition, Financial Data Analysis

2.11 Journal References

- Pérez-Ortega, J., Almanza-Ortega, N. N., Vega-Villalobos, A., Pazos-Rangel, R., Zavala-Díaz, C., & Martínez-Rebollar, A. (2019). The K-means algorithm evolution. In K. Sud, P. Erdogmus, & S. Kadry (Eds.), *Introduction to data science and machine learning* (Chap. 5). IntechOpen. <https://doi.org/10.5772/intechopen.85447>

Category 3- Principle Components (PCA)

3.1 Basics

Principal Component Analysis (PCA) is a method for taking a group of variables that tend to move together - several stock returns that rise and fall around the same time, for instance - and reducing them to a small number of combined measures that still capture most of what is happening, without needing to follow every original variable individually.

An analogy I find useful is three thermometers placed around the same room: each one is really reporting the same underlying room temperature, plus a bit of its own sensor noise. PCA works out, without being told, that three separate readings can be collapsed into one shared number plus some smaller residual detail. It is worth noting that PCA is unsupervised - there is no target variable being predicted here, only a search for repeated structure already present in the data.

3.2 Data & Setup

For this example, I simulated three stocks (Stock A, Stock B, Stock C) whose daily returns are driven by one hidden, shared 'market mood' factor plus each stock's own idiosyncratic noise. I chose this setup because it mirrors a common situation in finance, where several assets end up correlated simply because they are all responding to the same broad driver.

Table 3.1: Correlation between the three simulated stocks

	Stock A	Stock B	Stock C
Stock A	1.00	0.77	0.80
Stock B	0.77	1.00	0.78
Stock C	0.80	0.78	1.00

All three pairwise correlations turn out strongly positive, in the range of 0.77 to 0.80. This tells us that despite being three separate series, the stocks are largely repeating the same daily story - precisely the kind of redundancy PCA is designed to compress.

Before fitting PCA, I standardized all three return series to have a mean of 0 and a standard deviation of 1, so that no single stock could dominate the result purely because of differences in scale.

3.3 Running PCA

I ran PCA keeping all three possible factors, which makes it possible to see the full breakdown of where the variation goes rather than discarding information upfront.

Table 3.2: How much movement each factor explains

Factor	Explains on its own	Running total
6	85.6%	85.6%
Factor 2	7.7%	93.3%
Factor 3	6.7%	100.0%

A single combined factor already accounts for roughly 86% of the total movement across the three stocks, which suggests that three separate daily figures could reasonably be replaced by one, at the cost of losing only about 14% of the overall picture.

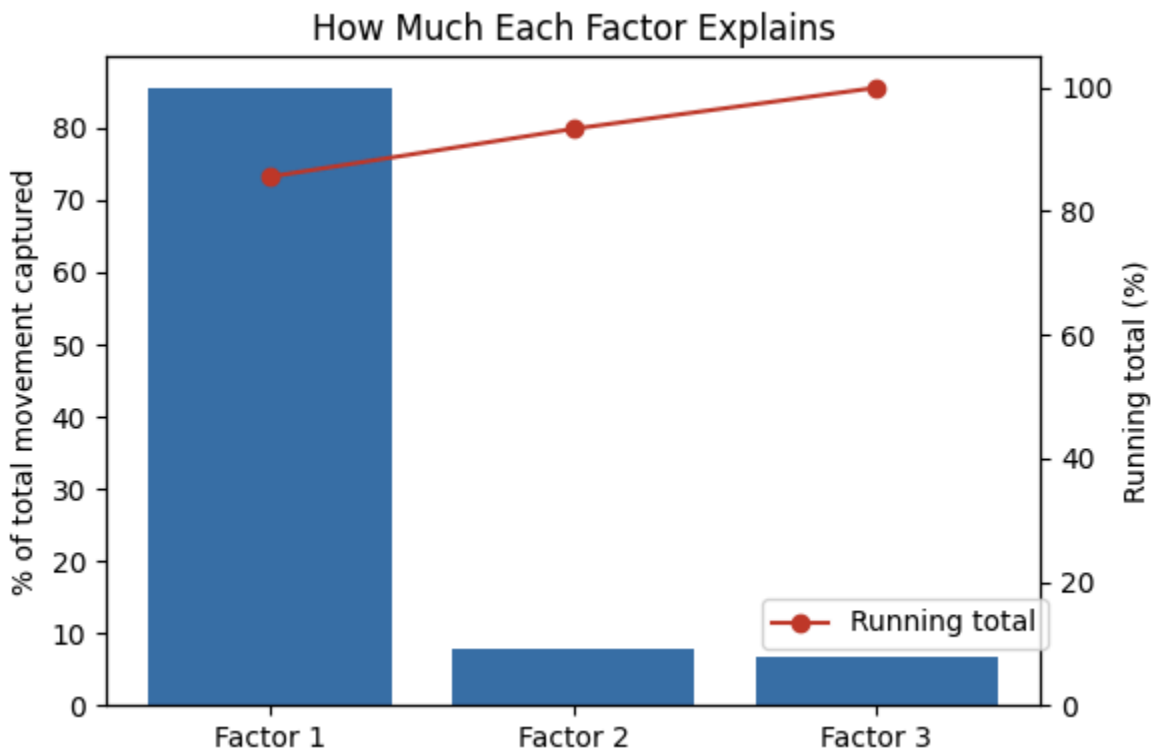


Figure 3.1: Scree plot- how much each factor explains, and the running total

3.4 What Factor 1 Represents

Table 3.3: Factor 1's recipe (its weight on each stock)

Stock A	Stock B	Stock C
0.58	0.57	0.58

Factor 1 assigns almost identical weight to all three stocks, which is consistent with the way the data was constructed: PCA rediscovered, without being told, that a single shared 'market mood' factor was driving all three series. The near-equal weighting is essentially what that underlying driver looks like once it has been extracted mathematically.

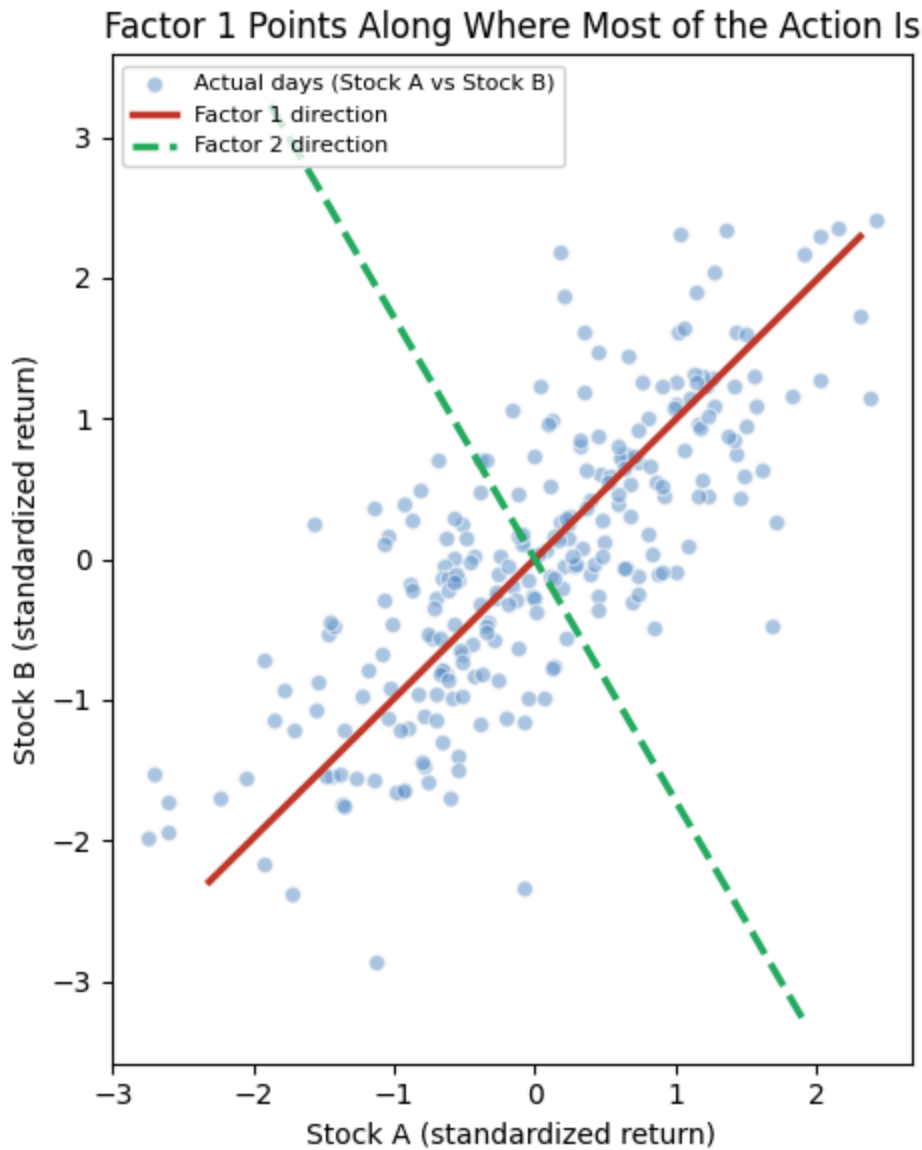


Figure 3.2: Stock A vs Stock B , with Factor 1's direction (solid) and Factor 2's direction (dashed) drawn through the data

As a check on this interpretation, I compared Factor 1's day-by-day score against a plain average of the three stocks. The two series track almost exactly the same path, with a correlation of 1.000 in this example, which supports the reading that Factor 1 is capturing what it appears to. In a less symmetric setting, PCA would offer more of an advantage over a simple average, since it finds the best combination of weights automatically rather than assuming the assets should be weighted equally.

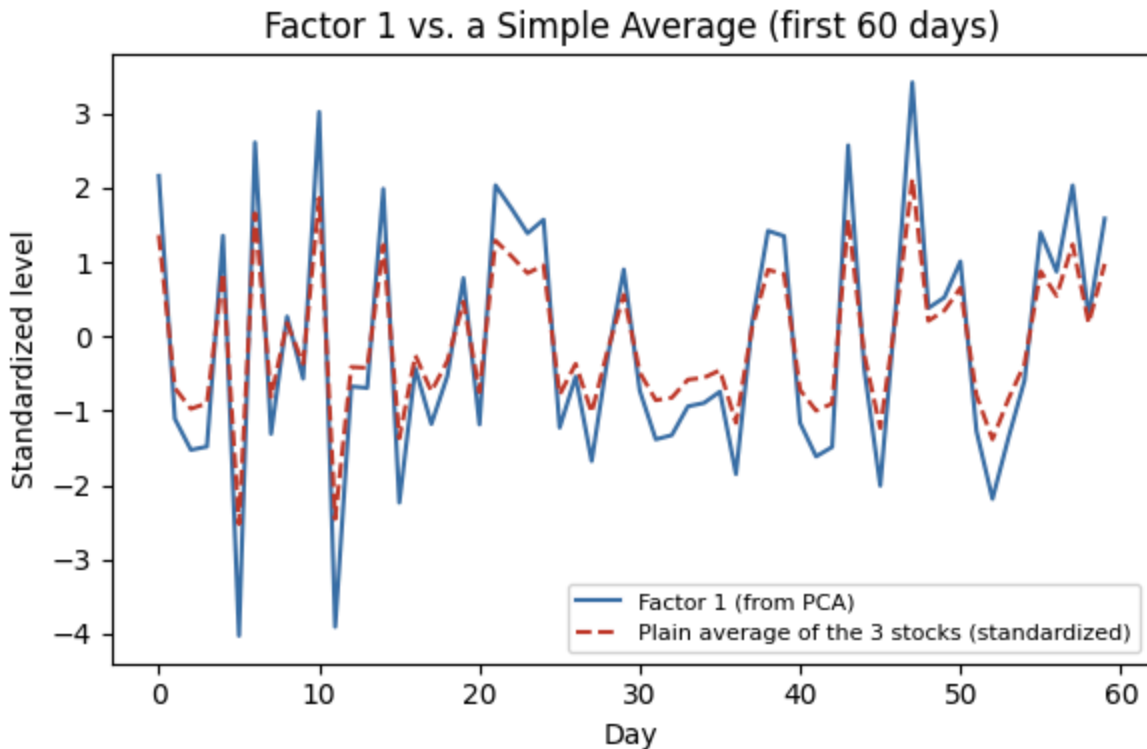


Figure 3.3: Factor 1 vs. A plain average of the 3 stocks, first 60 days

3.5 Illustration

Figures 3.1 through 3.3 together form the illustration for this method. Figure 3.1 (the scree plot) shows how much of the total movement each factor accounts for; Figure 3.2 overlays Factor 1 and Factor 2's directions on the actual data cloud, which is probably the clearest way to see what 'the direction of most spread' actually means geometrically; and Figure 3.3 provides the sanity check described above, comparing the compressed factor against a plain average.

3.6 Advantages

- Reduces clutter, since a handful of combined factors can often stand in for dozens of correlated variables.
- Removes the double-counting that occurs when several variables move together.
- Can reveal a hidden common driver, as the example above shows, without ever being told that driver exists.
- Produces factors that are uncorrelated with each other by construction, which tends to make later modelling steps more stable.

3.7 Disadvantages

- The factors produced are mathematical combinations rather than ready-made labels; sometimes, as in this example, the meaning is fairly obvious, but often it takes real digging to interpret what a factor represents.
- Sensitive to the scale of the input variables unless they are standardized beforehand.
- Only searches for straight-line directions of spread, so a curved relationship between variables can be missed entirely.
- One limitation is that compressing information always risks discarding a smaller factor that could still matter for a specific downstream question.

3.8 Equations

At its core, PCA repeatedly answers a single question: in which direction does the data spread out the most? Formally, this amounts to finding the eigenvectors and eigenvalues of the covariance matrix of the standardized data. The process can be summarized as follows:

- Compute how every variable relates to every other variable, captured by their covariance or correlation.
- Identify the single direction through the data cloud with the greatest spread; this becomes Factor 1.
- Find the next-best direction, orthogonal to the first, that captures as much of the remaining spread as possible; this becomes Factor 2, and the process continues.
- Each resulting factor is a specific weighted combination of the original variables, along with the percentage of total movement it explains.

3.9 Features

- Applies to any set of numeric variables that move together.
- Automatically ranks the factors from most to least important.
- Comes with a natural diagnostic for how many factors are worth keeping, namely the scree plot shown in Figure 3.1.

3.10 Guide: Inputs & Outputs

- **Inputs:** a table of numeric variables measured over the same time periods (here, three daily return series), typically standardized first.
- **Outputs:** a ranked set of factors, the share of movement each explains, each factor's weighting on the original variables, and a simplified version of the data expressed using just a few factors.

3.11 Hyperparameters

Table 3.4: PCA settings

Setting	What it controls
Number of components	How many factors to retain - fewer factors is simpler but explains less of the total movement
Standardize (yes/no)	Whether to rescale all variables to the same scale before fitting - in practice, almost always yes
Variance threshold	An alternative to fixing the number of factors directly: keep as many as are needed to reach a target share of variance explained, for example 90%

3.12 Keywords

Principal Component Analysis; PCA; Dimensionality Reduction; Unsupervised Learning; Variance Explained; Scree Plot; Factor Loadings; Standardization; Hidden Common Factor.

3.13 Journal References

Litterman, Robert, and Jose Scheinkman. "Common Factors Affecting Bond Returns." The Journal of Fixed Income, vol. 1, no. 1, 1991, pp. 54-61.

This is one of the best-known finance applications of PCA: Litterman and Scheinkman show that the seemingly complex, ever-changing shape of the interest rate (yield) curve across many maturities can be explained by a handful of underlying factors - broadly, its level, slope, and curvature - recovered using the same 'direction of most spread' logic applied in Section 3.3 above.

Category 4- Classification Trees

4.1 Basics

A classification tree predicts a category - Up or Down, in this example - by working through a short sequence of yes/no questions until it arrives at an answer. Each question splits the data into two groups, and the process repeats on each resulting group until the tree reaches a 'leaf', where it settles on a prediction. Trees belong to the CART (Classification and Regression Trees) family and, unlike a linear model, do not assume that the relationship between features and outcome follows a straight line.

4.2 Data & Setup

To set up a finance problem that at least looks realistic, even though the underlying data is simulated, I generated five separate assets using geometric Brownian motion with a mild autocorrelation term injected into the returns, so that price history carries a small, genuinely noisy amount of momentum. From these price paths I computed daily returns, moving-average deviations, rolling volatility, and 10-day momentum as features, then pooled all five assets into a single panel of 4,950 rows. The target was whether each asset's return on the following day was positive (Up) or negative (Down).

Table 4.1: Target balance in the pooled panel

Class	Share of days
Up	52.2%
Down	47.8%

I split the data by date rather than at random: the first 80% of trading days went into training, and the most recent 20% were held out purely for testing. This avoids the model effectively seeing the future during training, which a random split would risk.

4.3 Baseline Tree vs Tuned Tree

An untuned tree, grown without any depth limit, matched the training data almost perfectly but performed much worse on data it had not seen - a fairly textbook case of overfitting.

Table 4.2: Baseline (untuned) tree

Metric	Value
Train accuracy	1.00
Test accuracy	0.502
Tree depth grown	33
Number of leaves	966

I then tuned the hyperparameters using a systematic grid search over tree depth, minimum leaf size, and pruning strength, evaluated with a cross-validation scheme that respects time order, so that no fold is validated on data that predates its own training period.

Table 4.3: Tuned tree- best settings and test performance

Setting / Metric	Value
Best maximum depth	4
Best minimum leaf size	10
Best pruning strength (alpha)	0.0
Tree depth used	4
Number of leaves	15
Test accuracy	0.512
Test ROC-AUC	0.506

The signal here is genuinely weak, and I would treat that as the actual finding rather than a flaw in the approach: an ROC-AUC of about 0.51 is only marginally better than a coin flip, which is a realistic outcome when predicting short-horizon return direction from price-only technical features. Even so, pruning recovered a far more stable model - depth 4 with 15 leaves - compared with the untuned tree, even though its absolute accuracy remained close to random.

4.4 Confusion Matrix and Feature Importance

Confusion Matrix -- Tuned Classification Tree (Test Set)

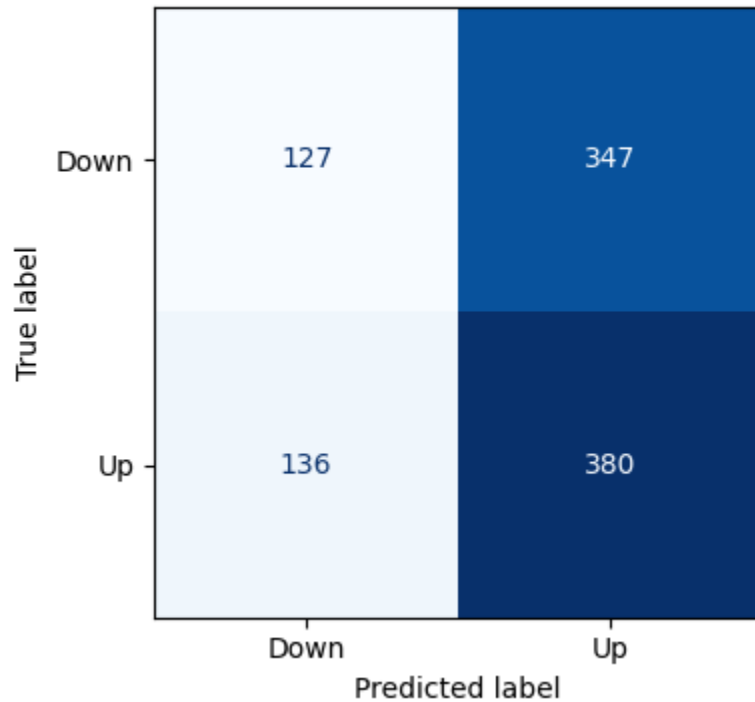


Figure 4.1: Confusion matrix, tuned tree, test set

The tuned model leaned toward predicting 'Up' more often than 'Down': it correctly identified 380 of the 516 actual Up days, but only 127 of the 474 actual Down days.

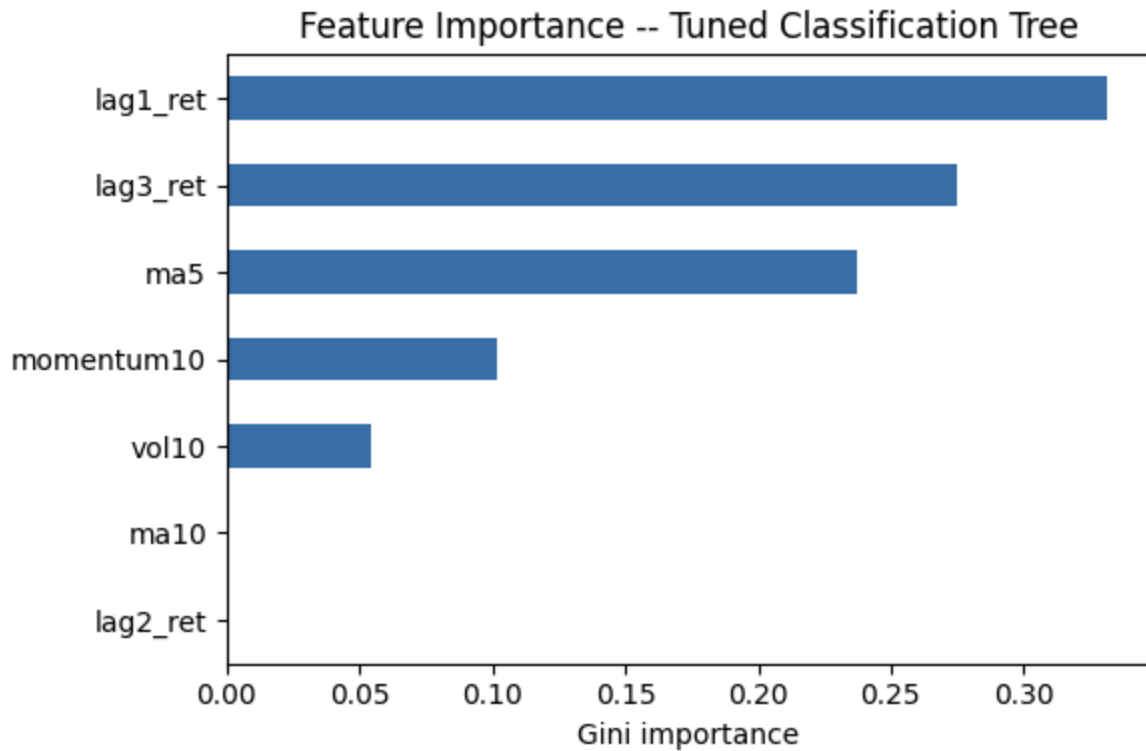


Figure 4.2: Feature importance, tuned tree

The most informative features were the 1-day and 3-day lagged returns together with the 5-day moving-average deviation; the 10-day moving average and the 2-day lagged return contributed nothing to the fitted tree, which suggests that whatever weak structure exists in this data operates over a very short horizon.

4.5 Illustration – the fitted Tree

Tuned Classification Tree (display truncated to depth 3 for readability)

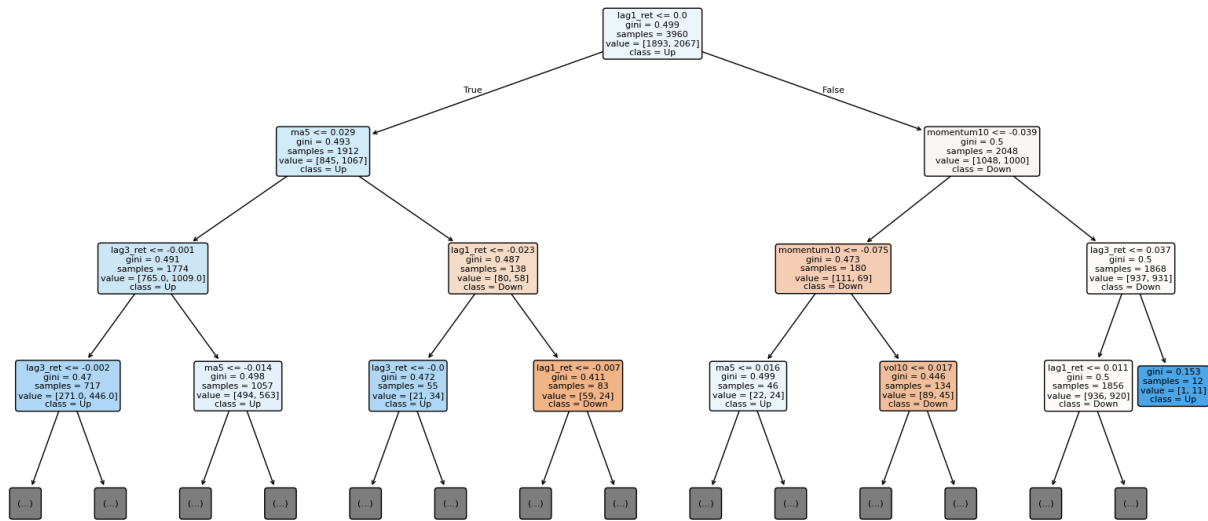


Figure 4.3: The tuned classification tree (full depth 4)

4.6 ROC Curve and Overfitting Check

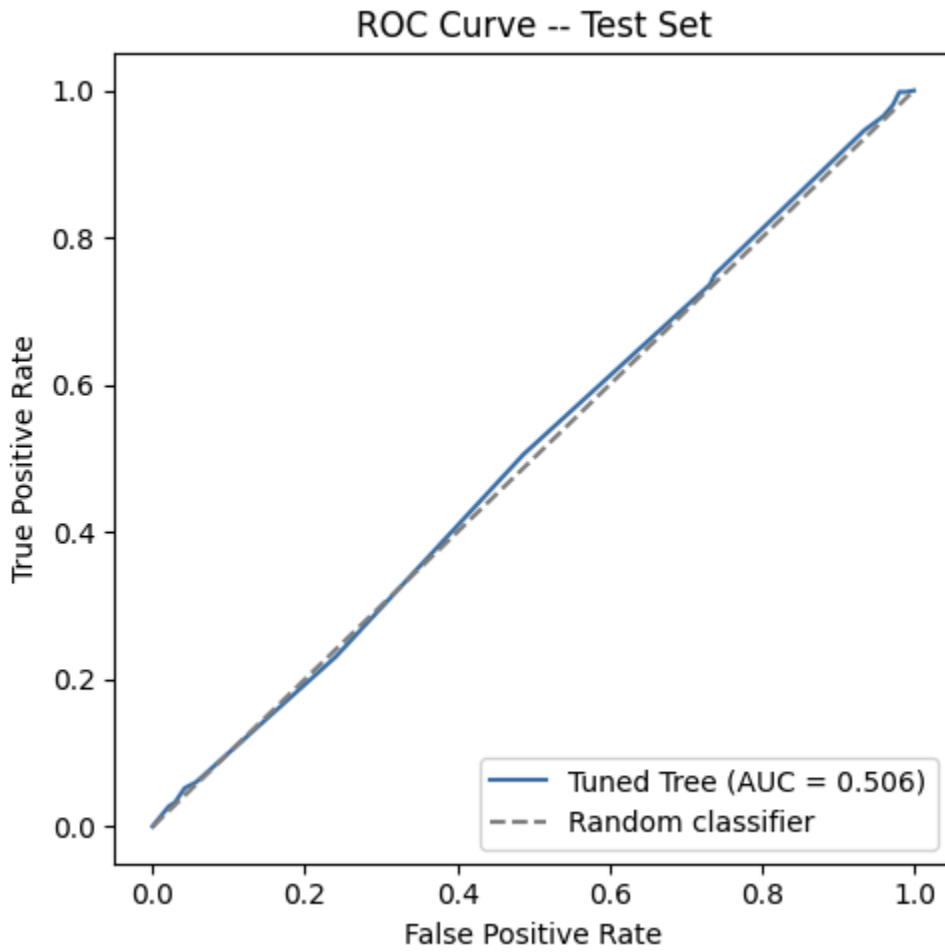


Figure 4.4: ROC curve, tuned tree, test set (AUC = 0.506)

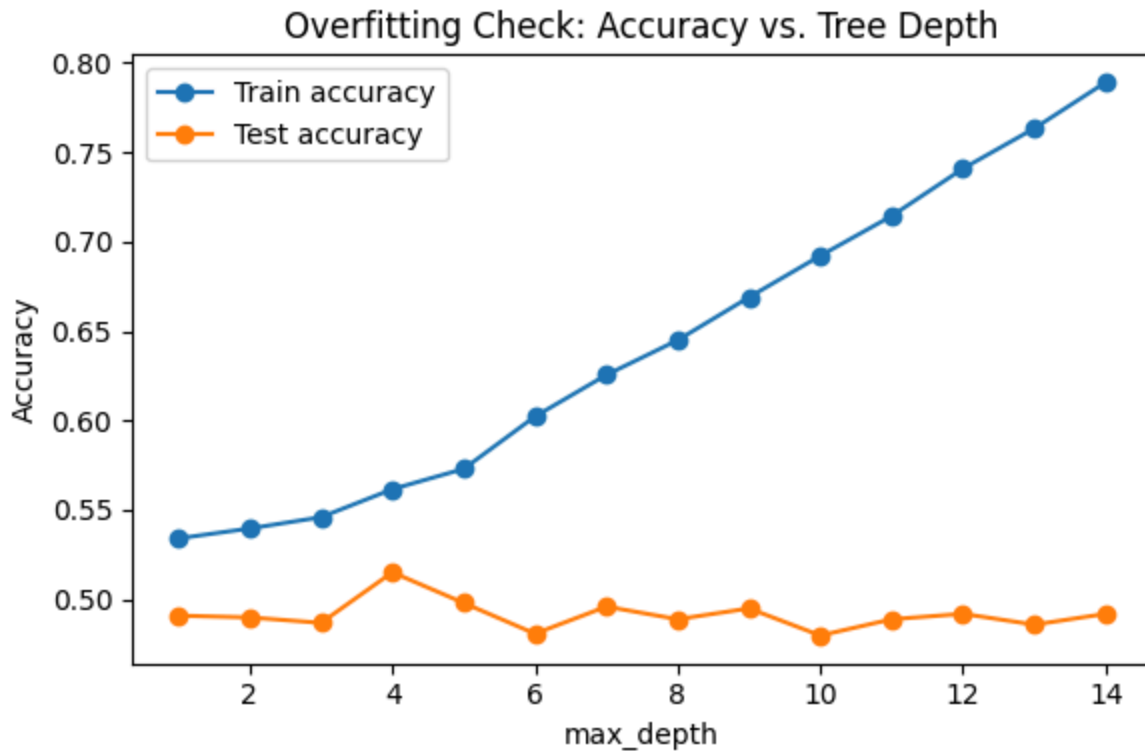


Figure 4.4: Train vs test accuracy as tree depth increase

Figure 4.5 is probably the clearest illustration of why maximum tree depth needs to be tuned rather than left at its default: training accuracy climbs steadily as depth increases, while test accuracy peaks early, around depth 4, and then drifts downward. Past that point the model is essentially memorizing noise rather than learning anything transferable.

4.7 Advantages

- Easy to read: a shallow tree is simply a sequence of if/then rules that a non-technical audience, such as a risk committee or a client, can follow directly.
- Requires no distributional assumptions and no rescaling of features.
- Handles a mix of numeric and categorical features without additional encoding work.
- Automatically ignores unhelpful features, since they are simply never selected for a split.

4.8 Disadvantages

- Prone to overfitting if left unconstrained, as Table 4.2 demonstrates directly.
- Exhibits high variance: a small change in the training data can produce a noticeably different tree structure.

- Built greedily, so each split is locally optimal but not necessarily part of the globally best tree.
- One limitation is that a smooth or linear relationship, which a simpler model could capture in a single step, may require many splits to approximate.

4.9 Equations

At each node, the tree evaluates every candidate feature/threshold pair and selects whichever split reduces impurity the most. Gini impurity for a node with K classes is given by:

$$Gini = 1 - \sum_{k=1}^K p_k^2$$

where p_k is the proportion of observations in that node belonging to class k . A pure node, containing only one class, has $Gini = 0$, while a 50/50 mix has the highest possible value. At each node, the split chosen is the one that most reduces the weighted Gini impurity across the two resulting child nodes. To control overfitting, cost-complexity pruning minimizes $R(T) + \alpha * |T|$, where $R(T)$ is the tree's total misclassification cost, $|T|$ is the number of leaves, and α penalizes additional leaves that do not sufficiently improve the fit.

4.10 Features

A tree handles numeric and categorical features side by side, with none of the one-hot encoding that other methods (like the SVM or LDA above) tend to require. It also comes with a feature-importance ranking built in for free — no extra work needed to get the chart shown in Figure 4.2. Because the whole thing is just a sequence of yes/no questions, it can be drawn out directly as a flowchart, which is exactly what Figure 4.3 shows. And beyond standing on its own, a single tree like this is also the basic building block behind more powerful ensemble methods — Random Forests and Gradient Boosted Trees both work by combining many trees like this one together.

4.11 Guide: Inputs and Outputs

- **Inputs:** a table of features — here, lagged returns, moving averages, rolling volatility, and momentum — plus a categorical target telling the model what actually happened (Up or Down).

- **Outputs:** the fitted tree itself, a predicted class for any new observation, the probability behind that prediction, and a ranking of which features the tree actually leaned on (shown earlier in Figure 4.2).

4.12 Hyperparameters

Table 4.4: Classification tree settings

Setting	What it controls
Maximum depth	How many splits deep the tree is allowed to grow from root to leaf — the main lever for keeping overfitting in check
Minimum leaf size	The smallest number of observations a leaf is allowed to have — stops the tree from carving out overly specific, one-off rules
Minimum split size	How many observations a node needs before it's even allowed to split again
Splitting criterion	Which impurity measure gets used to judge splits — Gini or entropy
Pruning strength (alpha)	The cost-complexity penalty applied during pruning — push this higher and the tree gets pruned back more aggressively

Depth is really the one that matter most in practice- its what turned the intined, 33-level deep tree from Table 4.2 into the far more stable depth-4 version used for final results.

4.13 Keywords

Classification Tree; CART; Decision Tree; Gini Impurity; Recursive Partitioning; Cost-Complexity Pruning; Feature Importance; Overfitting; Directional Prediction.

4.13 Journal Reference

Syed Nor, Shafinar, Sharifah Heryati Ismail, and Bee Wah Yap. "Personal Bankruptcy Prediction Using Decision Tree Model." Journal of Economics, Finance and Administrative Science, vol. 24, no. 47, 2019, pp. 157-170.

Syed Nor, Ismail, and Yap apply a decision-tree classification model to predict individual bankruptcy risk from demographic and financial variables and address the class-imbalance problem through random undersampling - a direct finance application of the same splitting-and-pruning mechanics used in Section 4.3 above.

Category 5– Linear Discriminant Analysis (LDA)

5.1 Basics Definition & Classification

LDA is a classic supervised classifier, and it works a bit differently from the tree or the SVM covered earlier: instead of just drawing a boundary between classes, it actually assumes something about how the data was generated. Specifically, it assumes each class is drawn from a normal distribution, and that all classes share the same "shape" of spread — one common covariance matrix. Once you make that assumption, Bayes' theorem hands you a decision rule for free, and that rule happens to be a straight line, which is where the name comes from.

Fisher's original way of framing it is a bit more intuitive. Rather than thinking in terms of probability distributions, he asked: what's the one direction you could project all this data onto so the classes separate as cleanly as possible? That's the same answer arrived at from a different angle. Here I'm using LDA as an interpretable baseline for the next-day up/down prediction problem.

5.2 Advantages

The main appeal of LDA is speed and simplicity. There's no iterative optimization involved — the class means, the shared covariance, and the priors are all estimated directly — so fitting it is basically instant and always gives the same answer, which isn't true of most other methods here. Because it pools all classes into one covariance estimate, it also uses limited data efficiently, which matters when the labeled financial history available isn't huge.

The discriminant weights are easy to read too: each one tells you directly how much a given feature pushes the prediction toward "up" or "down," and by how much. That kind of transparency is exactly what a risk committee or a compliance team wants to see before signing off on a signal. LDA also generalizes naturally to more than two classes and comes with calibrated posterior probabilities, which is handy if you ever wanted to size a position based on model confidence rather than just taking the binary call. As a side benefit, it can double as a dimensionality-reduction tool, since the same projection that separates classes also compresses the feature space.

5.3 Disadvantages

The catch is that all of this convenience rests on assumptions markets don't really satisfy. A single straight boundary can't capture non-linear or interacting effects, which caps how much accuracy is even possible. Returns are also not well-behaved in the way LDA assumes — they're fat-tailed and volatility comes in clusters rather than staying constant — so the "one shared covariance" assumption is shaky at best; when classes genuinely have different covariances, a related method (QDA) tends to do better.

LDA is also sensitive to outliers and to correlated features: a few extreme moves can distort the estimated mean and covariance, and correlated inputs make the covariance matrix hard to invert reliably. It needs more observations than features to get a stable covariance estimate in the first place, and if that's not the case, some form of shrinkage becomes necessary. Finally, like most of the methods here, it has no built-in way of handling missing values — everything needs to be cleaned or imputed first.

5.4 Equations

1. Posterior via Bayes' theorem — probability that observation x belongs to class k , with prior π_k and class density f_k :

$$P(y_k | x) = \frac{\pi_k f_k(x)}{\sum_l \pi_l f_l(x)}$$

2. Shared-covariance Gaussian class density:

$$f_k(x) = \frac{1}{(2\pi)^{\frac{p}{2}} |\Sigma|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(x - \mu_k)^\top \Sigma^{-1}(x - \mu_k)\right)$$

3. Linear discriminant function — taking logs and dropping terms constant across classes gives a function linear in x :

$$\delta_k(x) = x^\top \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^\top \Sigma^{-1} \mu_k + \ln \pi_k$$

4. Decision rule — assign x to the class with the largest discriminant score:

$$\hat{y} = \arg \max_k \delta_k(x)$$

Fisher's criterion. This is the projection view mentioned in Section 5.1 — find the direction w that separates the class means as much as possible relative to how spread out each class is internally:

$$w^* = \arg \max_w \frac{w^\top S_B w}{w^\top S_W w}$$

5.5 Features of the Model

LDA is linear, generative, and parametric, and unlike PCA or K-Means it needs labeled data to work — it's a supervised method through and through. Every boundary it draws between two classes comes out as a flat hyperplane; there's no way to curve it.

It won't tolerate missing values on its own, so the data needs to be complete or imputed before fitting. Scaling isn't strictly required the way it is for K-Means or SVM, but standardizing the features first still helps — it makes the weights easier to read and keeps the covariance matrix better behaved numerically. The model also assumes the inputs are roughly continuous and Gaussian-shaped, which is worth keeping in mind given how return data actually behaves.

On the output side, it hands back posterior probabilities rather than just hard labels, and it extends to more than two classes without any extra work. And because the whole thing has one exact closed-form solution, there's no randomness in training at all — run it twice on the same data and you get exactly the same answer both times.

5.6 Guide — Inputs & Outputs

Inputs: a numeric feature matrix (n samples, p continuous features) and a label vector telling the model which class each observation actually belonged to. Class priors and a shrinkage setting can optionally be supplied if the covariance estimate needs stabilizing.

Outputs: predicted labels, the probability behind each prediction, the fitted discriminant weights and intercept, and — if it's being used for dimensionality reduction rather than classification — a lower-dimensional projection of the data (capped at $K-1$ dimensions for K classes).

5.7 Hyperparameters

LDA doesn't ask for much tuning compared to the tree or the neural network, which is honestly a big part of its appeal as a baseline:

- Solver — svd by default; lsqr and eigen are alternatives that also support shrinkage.
- Shrinkage — regularizes the covariance estimate somewhere between 0 and 1, or it can be set automatically through Ledoit–Wolf; matters most when there isn't a lot of data relative to the number of features.
- Priors — the assumed probability of each class; defaults to whatever the actual class frequencies are in the training data.
- Number of components — only relevant if LDA is being used to reduce dimensions rather than classify; capped at $K-1$.

The specific choices used for tuning in this project are covered separately in the group Technical Section.

5.8 Illustration — How the Model Works

The essence of LDA is projecting the data onto a single axis chosen so the classes separate as cleanly as possible; the classifier then splits that axis at a threshold.

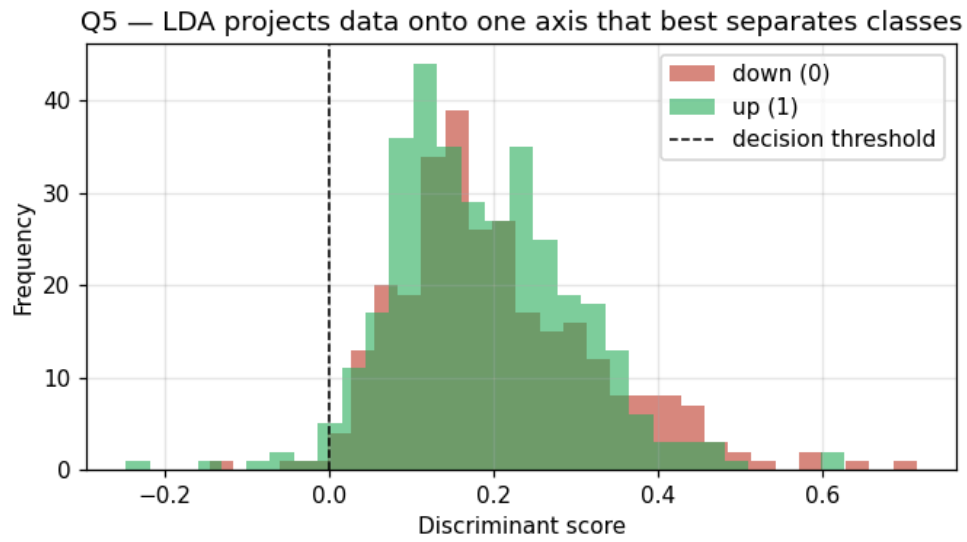


Fig. 5.1 — LDA collapses the multivariate features onto one discriminant axis; the dashed line is the classification threshold.

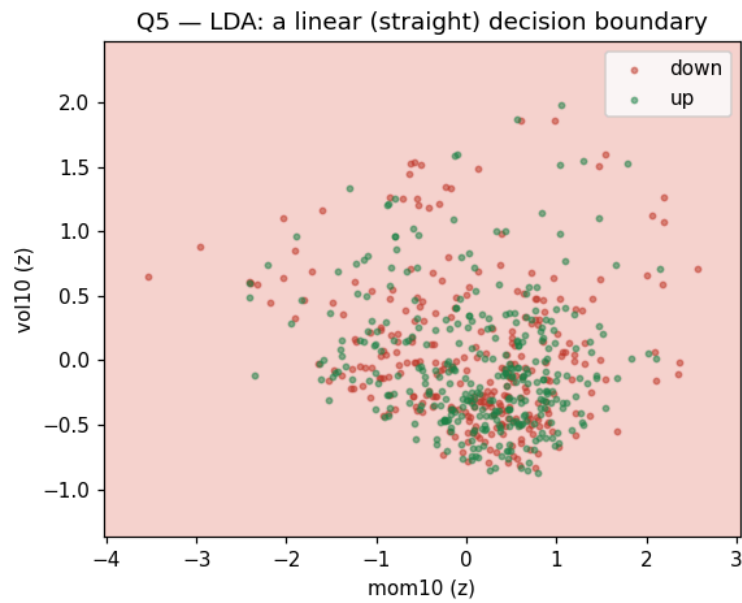


Fig. 5.2 — LDA yields a linear decision boundary (momentum vs. volatility).

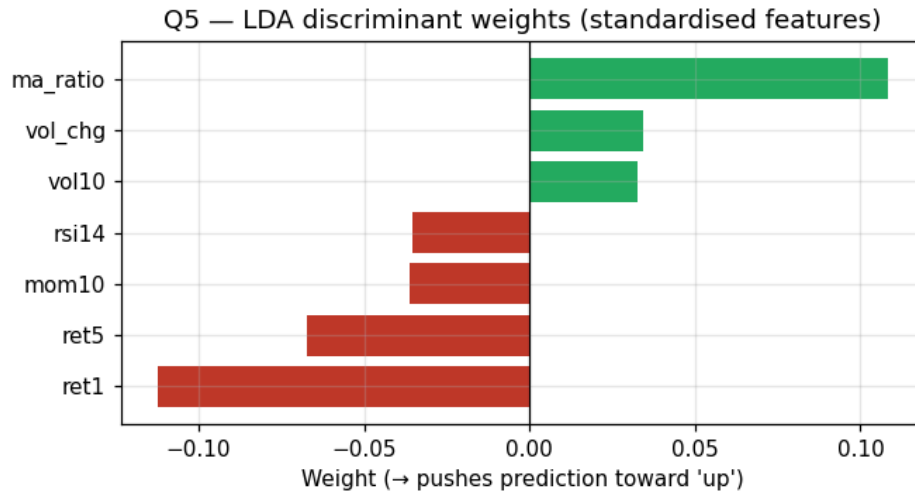


Fig. 5.3 — Estimated discriminant weights: sign and magnitude give each feature's contribution to the 'up' vote.

5.9 Computation (see accompanying notebook)

The notebook fits LDA (SVD solver) on the standardized training window and tests it on the chronologically later window, so nothing from the future leaks into training. On real SPY data, this landed at **53.5% test accuracy** and an **ROC-AUC of 0.47** — essentially a coin flip.

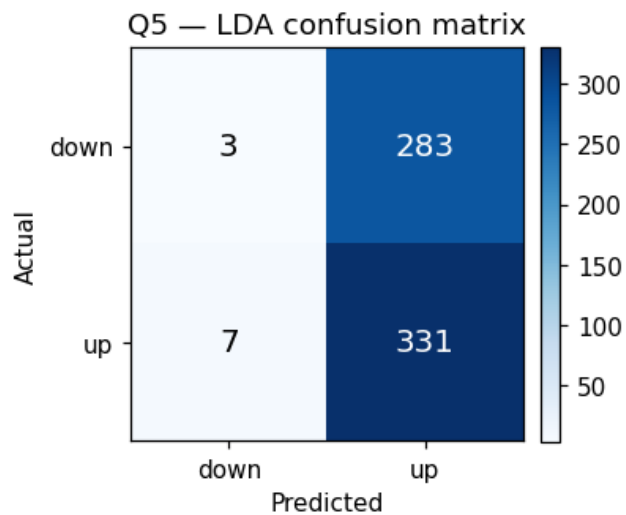


Fig. 5.4 — LDA confusion matrix (test set).

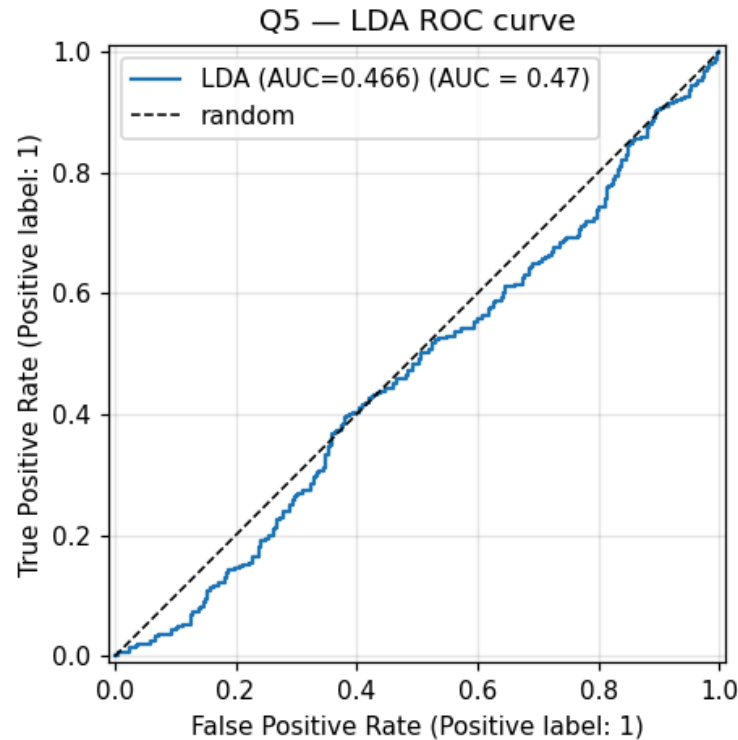


Fig. 5.5 — LDA ROC curve (test set).

Interpretation. An AUC that close to 0.5 on daily data isn't really a failure of the method — it's more evidence that next-day direction is close to unpredictable from price alone, which is roughly what you'd expect in a fairly efficient market. I'd treat LDA here as a fast, honest, transparent baseline rather than something to trade on directly; it's worth revisiting with richer features or a longer prediction horizon before any capital gets attached to it.

5.10 Journal Reference

Altman, Edward I. "Financial Ratios, Discriminant Analysis and the Prediction of Corporate Bankruptcy." *The Journal of Finance*, vol. 23, no. 4, 1968, pp. 589–609. — The seminal application of (multiple) discriminant analysis to finance, introducing the Z-score model for predicting corporate default.

Category 6– Support Vector Machine (SVM)

6.1 Basics

A Support Vector Machine looks for the best possible dividing line between two groups - specifically, the line that leaves the widest possible gap, or margin, between them, rather than settling for any line that happens to separate the data.

6.2 Part 1 : A Straight-Line Ex Credit Risk

The first example predicts whether a company is High risk or Low risk of default, using two indicators: its leverage ratio (debt relative to equity) and its profit margin. I simulated 160 companies according to a simple underlying rule - high leverage combined with a low margin tending toward high risk - plus some randomness, then split the sample 70/30 into training and test sets.

Table 6.1: Class balance in the simulated dataset

Class	Count
Low risk	87
High risk	73

Table 6.2: Linear SVM performance

Metric	Value
Training companies	112
Test companies	48
Train accuracy	0.88
Test accuracy	0.90
Number of support vectors	31

The model correctly classified 90% of companies it had not seen during training. Of the 112 training companies, 31 turned out to be support vectors - the borderline cases close enough to the boundary to actually influence where the line was drawn.

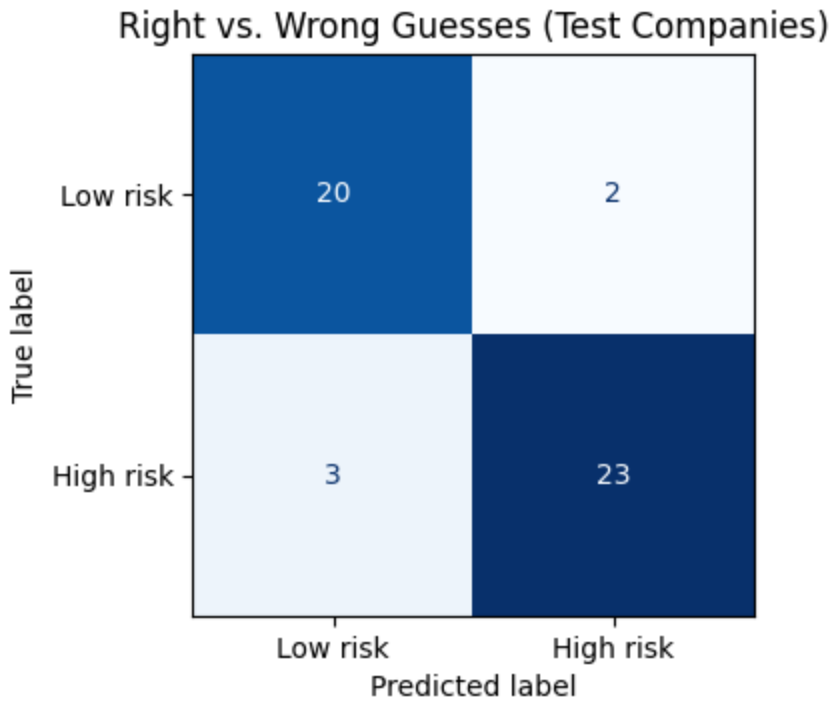


Figure 6.1: Confusion matrix, linear SVM, test set

6.3 Illustration- Dividing Line, Margin and Support Vectors

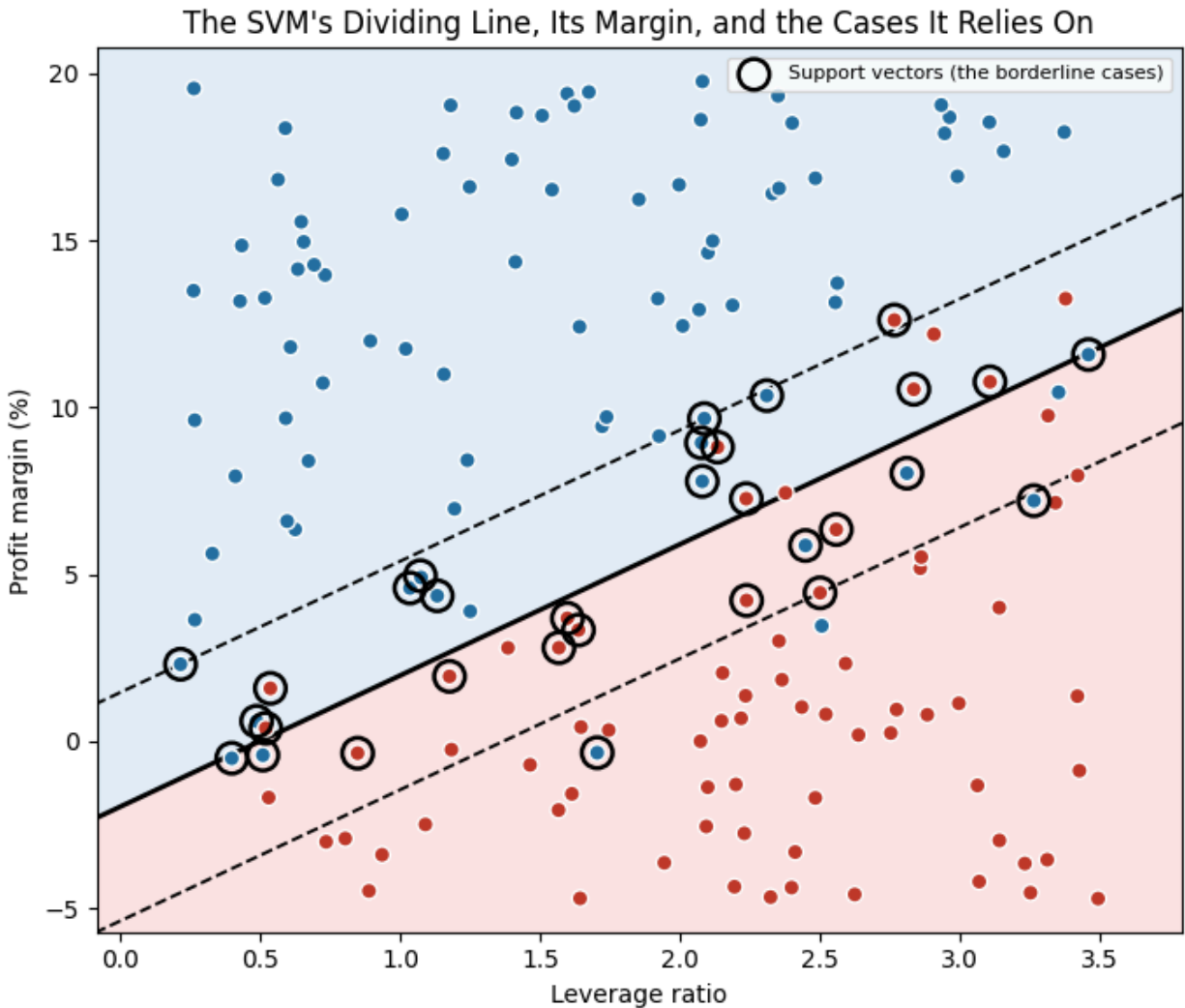


Figure 6.2: The SVM's dividing line (solid), its margin (Dashed), and the support vectors it relies on (circled)

The solid line is the SVM's decision boundary, and the two dashed lines mark the margin on either side of it. The circled points are the support vectors sitting right at the edge of that margin; every other point in the dataset could, in principle, be removed without the boundary shifting at all.

6.4 Part 2 – The Kernel Trick

To illustrate what happens when a straight line genuinely cannot separate two groups, I built a second, deliberately more difficult dataset: a 'Stress' market regime sitting inside a ring that is

completely surrounded by a 'Normal' regime. No straight line is capable of wrapping around a ring.

Table 6.3: Linear vs. Curved (RBF) kernel on ring-shaped data

Kernel	Accuracy
Linear (straight line)	0.62
RBF (curved)	0.99

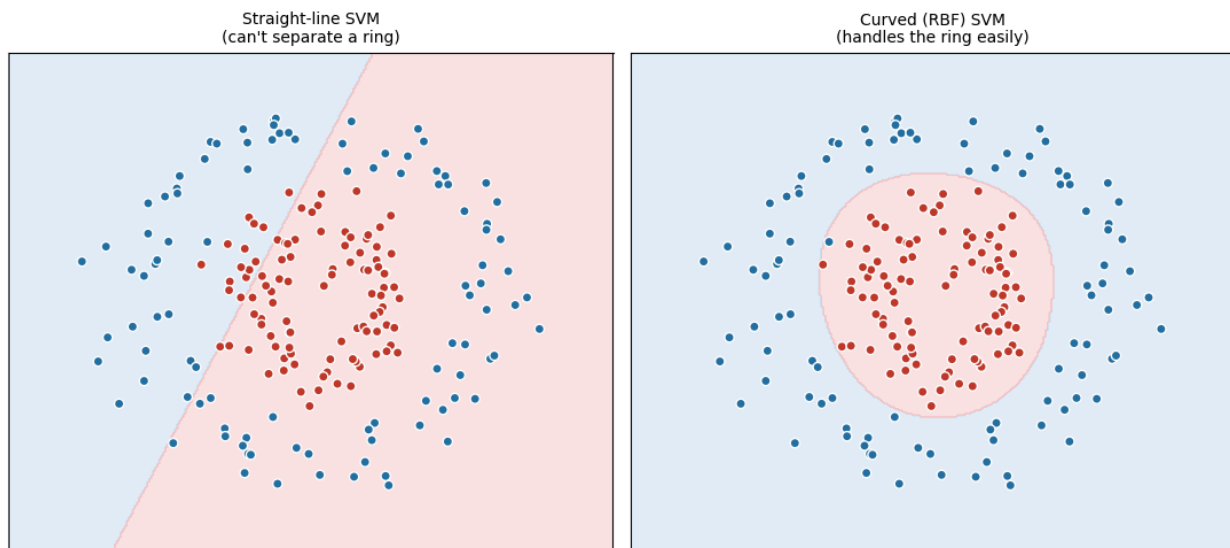


Figure 6.3: Straight-line SVM(left) vs curved RBF-kernel SVM (right) on the same ring-shaped data

The straight-line version barely outperforms a coin flip, at 62% accuracy, because it is structurally unable to bend around the ring. Switching only the kernel to RBF, with nothing else changed, lifts accuracy to 99%. This result is essentially the point of the kernel trick: the same underlying method, just given a more flexible boundary shape to work with.

6.5 Advantages

- Finds the most defensible boundary available - the widest-margin line rather than any line that happens to work - which tends to generalize better to new data.
- The boundary depends only on the support vectors, not on the overall volume of data.
- Extends from simple straight-line problems to more complex curved ones simply by changing the kernel.
- Reasonably resistant to a small number of outliers far from the boundary.

6.6 Disadvantages

- Training slows down noticeably as the dataset grows very large.
- Choosing an appropriate kernel and its settings, such as C and gamma, usually involves some trial and error.
- Less naturally interpretable than a tree, since a curved boundary is difficult to explain in intuitive terms.
- Sensitive to feature scale, since the method is fundamentally about distances and margins.

6.7 Equations

For a straight-line SVM, the dividing line is given by $w \cdot x + b = 0$, where x is a data point, w is a set of weights describing the line's orientation, and b shifts the line left or right. The margin width works out to $2 / ||w||$, so maximizing the margin is equivalent to minimizing $||w||$ subject to every point remaining on the correct side of the boundary. When a curved boundary is required, the kernel trick replaces the plain dot-product similarity between two points with an alternative similarity measure - the RBF kernel, for example, measures how close two points are - which has the effect of bending the boundary without the model ever explicitly computing anything in a higher-dimensional space.

6.8 Features

- Produces a compact set of support vectors summarizing exactly what the decision depends on.
- Comes with an adjustable strictness setting (C) trading off margin width against misclassified points.
- Moves seamlessly between straight-line and curved boundaries by changing only the kernel.

6.9 Guide: Inputs and Outputs

- **Inputs:** numeric features for each example (here, leverage ratio and profit margin), the correct category for historical examples, and a choice of kernel.
- **Outputs:** a predicted category for new data, a distance-from-boundary score usable as a rough confidence measure, and the specific support vectors defining the boundary.

6.10 Hyperparameters

Table 6.4: SVM settings

Setting	What it controls
C	How strict the model is about misclassified points - small C allows a wider margin with a few errors, large C insists on fewer errors with a narrower margin
kernel	Straight line (linear) or curved (rbf, poly, and others)
gamma	How far a single training point's influence reaches, for curved kernels - small values give smoother boundaries, large values risk overfitting

6.11 Keywords

Support Vector Machine; SVM; Margin; Support Vectors; Kernel Trick; Linear Kernel; RBF Kernel; Hyperplane; Credit Risk Classification.

6.12 Journal Reference

Bellotti, Tony, and Jonathan Crook. "Support Vector Machines for Credit Scoring and Discovery of Significant Features." *Expert Systems with Applications*, vol. 36, no. 2, 2009, pp. 3302-3308. *Bellotti and Crook use a Support Vector Machine to predict which loan applicants were likely to default, and examine which financial features mattered most to that decision - a direct finance application of the same widest-margin boundary approach used in Section 6.2 above.*

Category 7 — Neural Networks (Multi-Layer Perceptron)

7.1 Basics — Definition & Classification

A neural network — specifically a feed-forward multi-layer perceptron (MLP) here — is really LDA's more flexible cousin. Instead of drawing one straight boundary, it stacks layers of simple units ("neurons") on top of

each other. Each unit takes a weighted sum of its inputs, adds a bias term, and passes the result through a non-linear function such as ReLU. Stack enough of these layers together and, in theory, the network can approximate almost any relationship between inputs and outputs, however curved or tangled it is — that's the idea behind the universal approximation theorem.

The network learns through back-propagation: it checks how wrong a prediction was, then works backward through the layers nudging every weight slightly in whatever direction would have reduced that error, repeating this over many passes through the data. Here it's used as a more flexible alternative to LDA for the same next-day market direction problem.

7.2 Advantages

- The big thing a neural network can do that LDA can't is pick up on non-linear relationships and interactions between features. Markets don't always move in straight lines, so having a model that isn't restricted to one is genuinely useful. Given enough hidden units and enough data, a network can in theory represent almost any input-output relationship — that's the universal approximation idea mentioned above. It also builds its own internal features as it trains, so there's less manual work figuring out which combinations of variables might matter.
-
- Beyond that, the same basic architecture scales up in a lot of directions. Add recurrence and you get something built for sequences (RNNs, LSTMs); add convolutions and you get something built for images (CNNs). None of that changes the core idea — layers of weighted sums and non-linear activations — it just adapts the shape to the problem. And unlike LDA, which tends to plateau once you throw more data at it, a network usually keeps getting better as the dataset grows.

7.3 Disadvantages

The obvious cost of all that flexibility is data. Networks generally need a lot of it to train well, and financial data tends to be limited, noisy, and non-stationary (meaning the underlying relationships shift over time) — not exactly ideal conditions. Overfitting is a real risk here for exactly that reason.

There's also the black-box problem. LDA's weights tell you directly how each feature pushes the prediction one way or the other; a trained network's weights don't really mean anything on their own. That's a genuine issue if the model needs to be explained to a regulator or a client. On top of that, training isn't as clean or predictable as it is with LDA — there are a lot of settings to get right, the loss surface has multiple valleys instead of one clear minimum, and results shift depending on how the weights were initialized. Even with the same seed, small implementation differences can move the result around. Add to that the fact that training just takes longer and needs more compute than LDA, plus real care with scaling and regularization (L2, dropout, early stopping) to keep it from memorizing the training data outright.

7.4 Equations

Forward pass (one layer). The activations coming out of layer l depend on the activations from the layer before it, a weight matrix W , a bias term b , and whatever activation function g is being used:

$$a^{(l)} = g(W^{(l)} a^{(l-1)} + b^{(l)})$$

Activation functions. ReLU is used in the hidden layer; sigmoid is used at the output for binary classification:

$$g(z) = \max(0, z) \quad \hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Loss. Binary cross-entropy, with an L2 penalty (strength α) added to discourage the weights from growing too large:

$$\mathcal{L} = -\frac{1}{N} \sum_i \left[y_i \ln \hat{y}_i + (1 - y_i) \ln (1 - \hat{y}_i) \right] + \frac{\alpha}{2} \sum \|W\|^2$$

Back-propagation. The error term at layer l comes from the layer right after it, combined element-wise ($\odot$) with the derivative of the activation:

$$\delta^{(l)} = (W^{(l+1)\top} \delta^{(l+1)}) \odot g'(z^{(l)})$$

Weight update. Standard gradient descent, with learning rate η controlling the step size:

$$W \leftarrow W - \eta \frac{\partial \mathcal{L}}{\partial W}$$

7.5 Features of the Model

This is a non-linear, discriminative model, and — like the tree and the SVM before it — it's supervised, so it needs labeled outcomes to train on. Where it differs from LDA is the shape of its decision boundary: instead of a straight line, it can curve however the data needs it to, depending on how deep and wide the network is built.

A few practical notes: it doesn't handle missing values on its own, so those need to be dealt with beforehand. Feature scaling isn't optional here — gradient-based training just doesn't work well without it. Training is also stochastic, meaning the random starting weights and the order data gets fed in during mini-batches both affect the outcome, so a fixed seed matters if you want reproducible results. On the output side, it gives you class probabilities through sigmoid or softmax (or plain continuous numbers if it's set up for regression instead).

7.6 Guide — Inputs & Outputs

Inputs: a scaled feature matrix (n rows by p features) along with the target labels, plus whatever architecture and training settings get chosen (covered in the next section). The weights themselves start out randomly initialized before training begins.

Outputs: predicted labels and their associated probabilities, the final trained weights and biases across every layer, and some way of checking the training went okay — usually a plot of the loss curve over training.

7.7 Hyperparameters

There's a fair number of settings to get right with a network, more than with any other method in this report:

- **Architecture** — how many hidden layers there are and how many units sit in each one; this is basically the model's capacity.
- **Activation function** — ReLU was used here, though tanh or logistic are other common choices.
- **Regularization (α)** — L2 weight decay, sometimes paired with dropout.
- **Optimizer and learning rate** — Adam or SGD, along with how big each weight update step (η) is.
- **Batch size and number of epochs.**
- **Early stopping** — how many epochs to wait without improvement before halting training.

These were tuned in the notebook using a time-series cross-validated grid search (TimeSeriesSplit, scored on ROC-AUC), which respects the fact that this is sequential data rather than shuffling it randomly. The configuration that came out on top: a single hidden layer of 16 ReLU units, $\alpha = 1e-4$, and a learning rate of 0.01. These specific values feed into the group Technical Section.

7.8 Illustration — How the Model Works

A network learns by repeatedly nudging its weights to bring the loss down (Fig. 7.1). Its capacity needs to be tuned carefully — too few hidden units and it underfits, too many and it starts overfitting (Fig. 7.2). The actual payoff versus LDA shows up as a curved decision boundary instead of a straight one (Fig. 7.3).

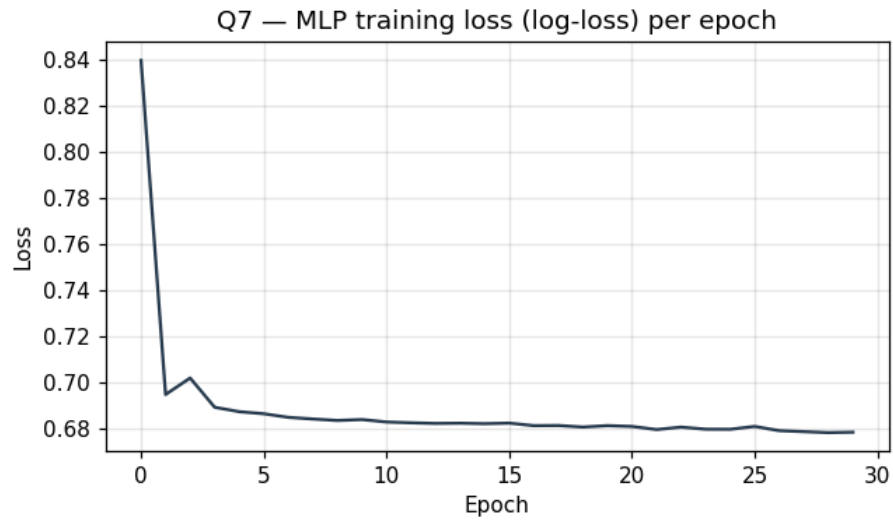


Fig. 7.1 — Training loss (log-loss) falls as back-propagation updates the weights each epoch.

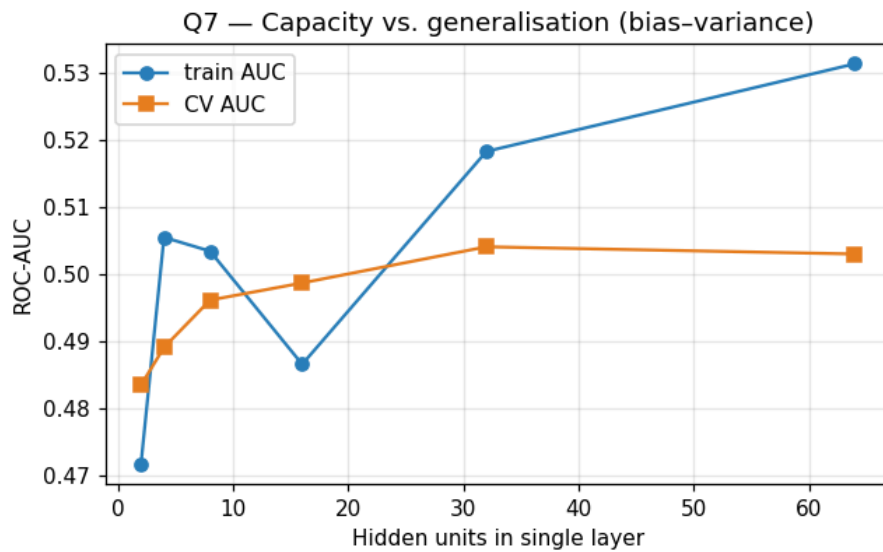


Fig. 7.2 — Validation curve: CV AUC vs. hidden-layer width makes the bias-variance trade-off explicit.

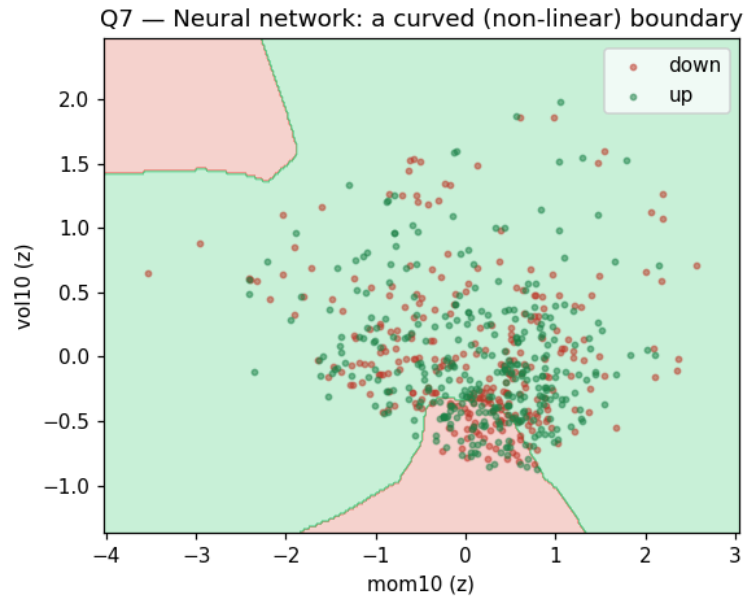


Fig. 7.3 — The network learns a non-linear decision boundary (same two features as Fig. 5.2), unlike LDA's straight line.

7.9 Computation (see accompanying notebook)

The tuned MLP was evaluated on the exact same held-out test window as LDA, for a fair comparison: **51.6% test accuracy** and an **ROC-AUC of 0.47**.

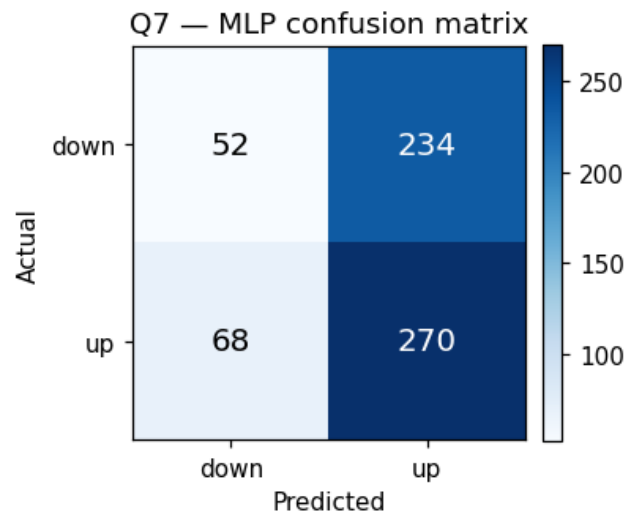


Fig. 7.4 — MLP confusion matrix (test set).

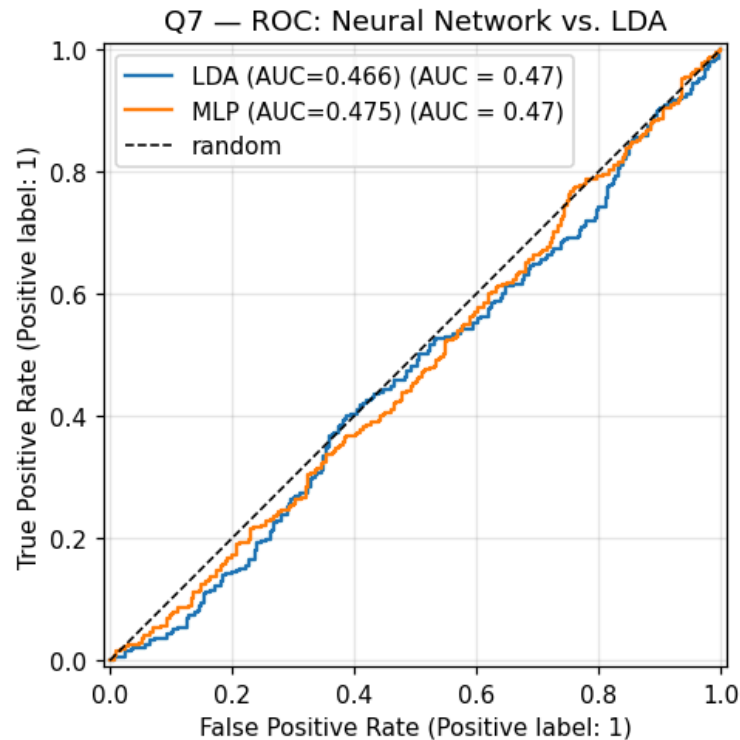


Fig. 7.5 — ROC comparison: neural network vs. the LDA baseline.

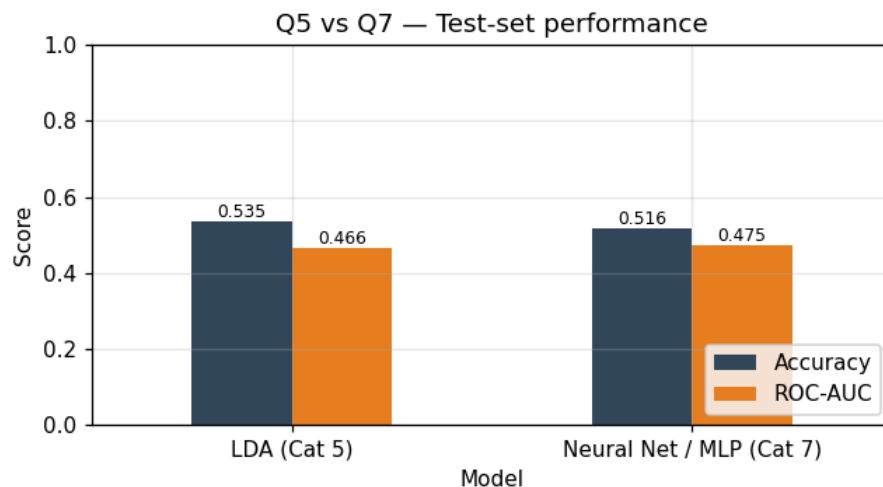


Fig. 7.6 — Test-set accuracy and ROC-AUC: LDA vs. MLP.

Interpretation. The network came out essentially tied with LDA — maybe a touch better on ranking quality (AUC), but nothing decisive. That's a fair outcome rather than a disappointing one: at a one-day horizon using only price-based features, there's probably just not much non-linear structure sitting there to be found, which again points toward markets being close to efficient at this timescale. I'd hold off on using neural networks here and instead save them for problems with richer inputs — higher-frequency data, more features, or alternative data sources — where that extra flexibility has something real to grab onto. LDA remains the more

useful benchmark precisely because it's simple enough to trust and cheap enough to keep re-running as conditions change.

7.10 Journal Reference

Krauss, Christopher, Xuan Anh Do, and Nicolas Huck. "Deep Neural Networks, Gradient-Boosted Trees, Random Forests: Statistical Arbitrage on the S&P 500." *European Journal of Operational Research*, vol. 259, no. 2, 2017, pp. 689–702. Krauss, Do, and Huck build a statistical-arbitrage strategy on S&P 500 stocks using deep neural networks alongside tree-based ensembles — a large-scale version of the same non-linear pattern-finding demonstrated on a much smaller scale in Section 7 above.

Technical Section – Hyperparameter Tuning

Ridge Regression

Ridge Regression involves setting the hyperparameter “alpha” to regulate the degree of regularization of the regression model. Alpha decides how strictly the value of large coefficients is penalized in the process of learning. This parameter is essential for managing multicollinearity and stabilizing the model. The alpha parameter in this project was tuned by considering several pre-defined values. The training was done using standardized features and time-series splitting.

K-means clustering

The main hyperparameter used in K-Means is K, which refers to the number of clusters created by the algorithm. Choosing an appropriate K value is very important as it will determine how the data points are clustered. One approach that can be used to choose an appropriate K is using the Elbow Method, in which K values are chosen, and the compactness of clustering is determined. Other hyperparameters used in K-Means include k-means++, n_init, and max_iter.

Marketing Alpha

Ridge Regression

Ridge Regression is one of the most popular forms of linear regression analysis where the independent variables in the model tend to be highly correlated. This method enhances stability through the reduction of coefficients. This form of regression analysis is particularly important

in financial modeling where macro-economic factors tend to be highly correlated, such as inflation rates, interest rates, and market indicators.

K-means clustering

K-means clustering is an unsupervised learning algorithm that segments data by placing them into different groups according to their similarities. The use of this algorithm in finance is widespread as it can be applied in analyzing market behavior, customer segmentation, and market structure. The strength of K-means clustering is its ability to analyze large volumes of data owing to its ease of use and scalability. It allows converting complicated data into groups, which enables informed decision making.

K-Means gave us something linear regression can't: structure we didn't ask for. Without being told anything about sectors or fundamentals, it split S&P 500 stocks into five groups that lined up with real risk-return profiles — defensive, high-risk, low-growth, neutral, and high-growth. That's a fast way to segment a large universe of stocks before doing anything else with them.

PCA's strength was compression. Three correlated simulated stocks collapsed into one factor that explained 85.6% of their movement, and that factor turned out to represent almost exactly the same thing as a simple average would — but PCA found that weighting on its own, without being told the stocks shared a common driver. In a less symmetric case, that's exactly where PCA has an edge over just averaging things by hand.

The Classification Tree's biggest strength wasn't accuracy — the signal in daily return direction was close to a coin flip no matter how the tree was tuned. Its strength was honesty and interpretability: a shallow tree is a sequence of if/then rules, which is something a non-technical audience like a risk committee can actually follow, and pruning made the overfitting problem visible and fixable.

LDA and the Neural Network were tested on the same problem and came out almost tied (53.5% vs. 51.6% accuracy, both around 0.47 AUC). That result itself is informative: adding a much more complex, less interpretable model didn't buy anything extra on this data, which argues for starting with the simple, transparent method (LDA) and only reaching for something like a neural network when there's a real reason to believe non-linear structure exists.

The SVM had the clearest "before and after" of any method in the report. On the credit risk example it hit 90% test accuracy with just two features. On the ring-shaped data, switching only the kernel from linear to RBF took accuracy from 62% to 99% — same method, same data, one setting changed. That's a strong argument for why the kernel trick matters.

Final Summary

- PCA compressed three correlated simulated stocks into a single factor capturing 85.6% of their combined movement, and that factor turned out to place almost equal weight on all three stocks - effectively rediscovering the shared 'market mood' driver used to construct the data, without being told it was there.
- The classification tree, once tuned to a maximum depth of 4 with 15 leaves, reached about 51% test accuracy predicting next-day return direction - only marginally better than chance, which I take as a realistic and honest result for this kind of short-horizon, price-only signal, and a clear demonstration of why pruning matters (the untuned tree scored 100% on training data but only 50% on test data).
- The SVM reached 90% test accuracy separating High-risk from Low-risk companies using only two features, and a second experiment demonstrated the value of the kernel trick directly: 62% accuracy with a straight-line kernel versus 99% with a curved (RBF) kernel on the same ring-shaped data.
- One limitation running through all three exercises is that none of these results should be treated as a standalone trading or lending signal - each uses simulated data and a small number of features, chosen deliberately to make the mechanics of the method clear rather than to produce a genuine market call.
- Consistent with the rest of the team's report, my recommended next step would be to re-run all three notebooks on real, current data before any of these methods are allowed to inform an actual decision.

References

Bellotti, Tony, and Jonathan Crook. "Support Vector Machines for Credit Scoring and Discovery of Significant Features." *Expert Systems with Applications*, vol. 36, no. 2, 2009, pp. 3302-3308.

Breiman, Leo, Jerome Friedman, Richard Olshen, and Charles Stone. *Classification and Regression Trees*. Wadsworth, 1984.

Cortes, Corinna, and Vladimir Vapnik. "Support-Vector Networks." *Machine Learning*, vol. 20, no. 3, 1995, pp. 273-297.

Litterman, Robert, and Jose Scheinkman. "Common Factors Affecting Bond Returns." *The Journal of Fixed Income*, vol. 1, no. 1, 1991, pp. 54-61.

Pearson, Karl. "On Lines and Planes of Closest Fit to Systems of Points in Space." *Philosophical Magazine*, vol. 2, no. 11, 1901, pp. 559-572.

Syed Nor, Shafinar, Sharifah Heryati Ismail, and Bee Wah Yap. "Personal Bankruptcy Prediction Using Decision Tree Model." *Journal of Economics, Finance and Administrative Science*, vol. 24, no. 47, 2019, pp. 157-170.

Nuria Alemany, Vicent Aragó, Enrique Salvador, Uncovering the risk-return trade-off through ridge regressions, *Finance Research Letters*, Volume 71, 2025, 106420, ISSN 1544-6123, <https://doi.org/10.1016/j.frl.2024.106420>.

Dai, Z., Li, T., & Yang, M. (2022). Forecasting stock return volatility: The role of shrinkage approaches in a data-rich environment. *Journal of Forecasting*, 41(5), 980–996.

<https://doi.org/10.1002/for.2841>

Pérez-Ortega, J., Almanza-Ortega, N. N., Vega-Villalobos, A., Pazos-Rangel, R., Zavala-Díaz, C., & Martínez-Rebollar, A. (2019). The *K*-means algorithm evolution. In K. Sud, P. Erdogmus, & S. Kadry (Eds.), *Introduction to data science and machine learning* (Chap. 5). IntechOpen. <https://doi.org/10.5772/intechopen.85447>

Learn More

Journal articles (from the individual sections above):

Alemany, Nuria, et al. "Uncovering the Risk-Return Trade-Off through Ridge Regressions." *Finance Research Letters*, vol. 71, 2025, article 106420, <https://doi.org/10.1016/j.frl.2024.106420>.

Altman, Edward I. "Financial Ratios, Discriminant Analysis and the Prediction of Corporate Bankruptcy." *The Journal of Finance*, vol. 23, no. 4, 1968, pp. 589-609.

Bellotti, Tony, and Jonathan Crook. "Support Vector Machines for Credit Scoring and Discovery of Significant Features." *Expert Systems with Applications*, vol. 36, no. 2, 2009, pp. 3302-3308.

Dai, Zhifeng, et al. "Forecasting Stock Return Volatility: The Role of Shrinkage Approaches in a Data-Rich Environment." *Journal of Forecasting*, vol. 41, no. 5, 2022, pp. 980-996, <https://doi.org/10.1002/for.2841>.

Group Number: 15968

Krauss, Christopher, et al. "Deep Neural Networks, Gradient-Boosted Trees, Random Forests: Statistical Arbitrage on the S&P 500." *European Journal of Operational Research*, vol. 259, no. 2, 2017, pp. 689-702.

Litterman, Robert, and Jose Scheinkman. "Common Factors Affecting Bond Returns." *The Journal of Fixed Income*, vol. 1, no. 1, 1991, pp. 54-61.

Pérez-Ortega, Joaquín, et al. "The K-Means Algorithm Evolution." *Introduction to Data Science and Machine Learning*, edited by Kenan Sud et al., IntechOpen, 2019, <https://doi.org/10.5772/intechopen.85447>.

Syed Nor, Shafinar, et al. "Personal Bankruptcy Prediction Using Decision Tree Model." *Journal of Economics, Finance and Administrative Science*, vol. 24, no. 47, 2019, pp. 157-170.

Additional websites (strengths-focussed):

"How Machine Learning Is Transforming the Investment Process." *CFA Institute*, www.cfainstitute.org/insights/articles/how-machine-learning-is-transforming-the-investment-process.

Doyle, Jordan. "How Machine Learning Is Transforming Portfolio Optimization." *CFA Institute Enterprising Investor*, 5 Sept. 2024, blogs.cfainstitute.org/investor/2024/09/05/how-machine-learning-is-transforming-portfolio-optimization/.

"What Is Machine Learning?" *IBM*, www.ibm.com/think/topics/machine-learning.

Comparing Models

Feature	Ridge Regression	K-Means	PCA	Classification Tree	LDA	SVM	Neural Network (MLP)
Supervised / Unsupervised	Supervised	No labels needed	No labels needed	Supervised	Supervised	Supervised	Supervised
Missing Data	Needs clean data going in	Same — no gaps allowed	Same	Doesn't tolerate NAs either	Requires complete data	Needs everything filled in	Also needs complete data
Feature Scaling	Required	Required (distance-based)	Required	Not needed — splits don't care about scale	Helps but isn't strictly required	Required — very scale-sensitive	Required for training to converge properly
Correlated / Multicollinear Inputs	This is exactly what it's built to fix	N/A for a clustering method	Actually turns correlation into an advantage	Copes okay, mostly ignores redundant features	Struggles a bit here	Distance gets distorted by it	Can work through it but isn't designed with this in mind
Non-Linear Relationships	Can't capture these — strictly linear	N/A	Only finds straight-line directions of spread	Yes, splits can approximate curves	No — one straight boundary only	Yes, once you switch to a kernel like RBF	Yes — this is really its main strength
How Easy to Interpret	Coefficients read directly, fairly clear	Cluster profiles are intuitive once labeled	Loadings take some digging to interpret	Very easy — literally a flowchart	Weights are directly readable	Hard to explain once curved	Basically a black box
Overfitting Tendency	Kept in check by the penalty term	N/A	N/A	Bad if left unpruned, as shown earlier	Naturally low	Depends heavily on C and kernel choice	High risk without regularization
Training Time	Fast, closed-form	Fast	Fast	Fast	Fastest of all seven — no iteration needed	Slows down a lot on bigger datasets	Slowest — many epochs needed